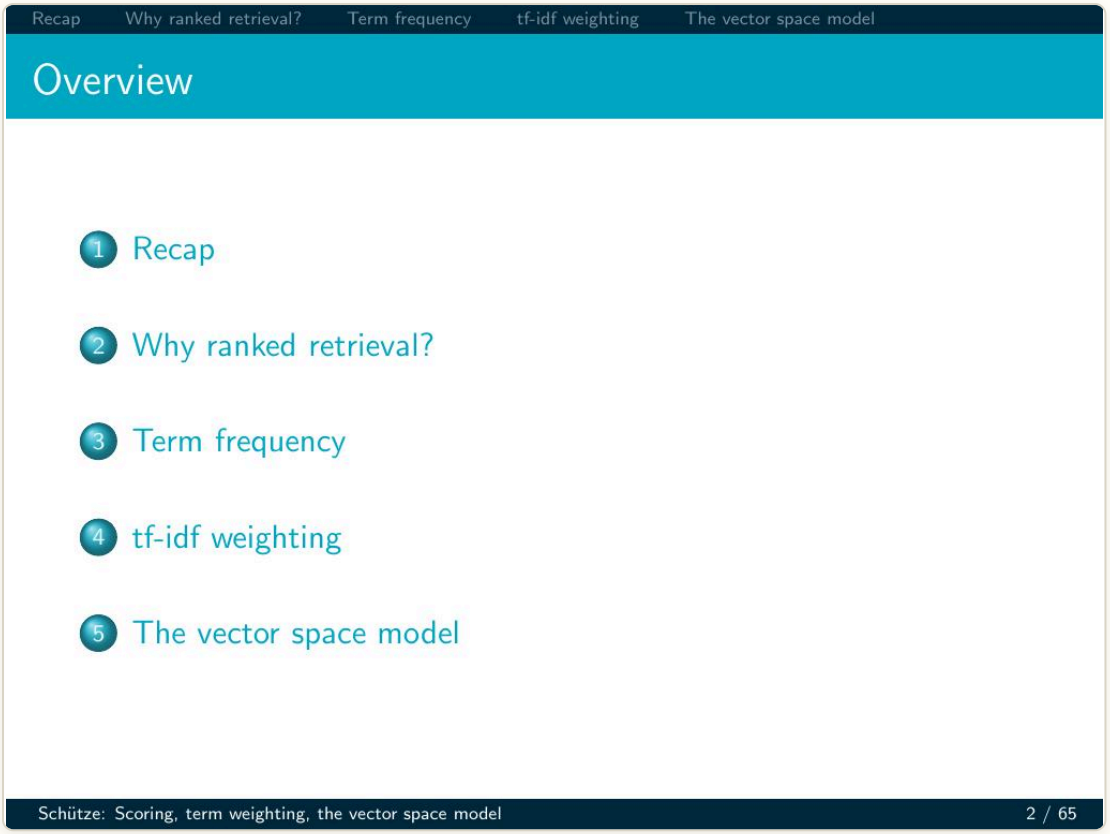


정보검색 입문 — IIR 6: 점수화, 단어 가중치, 벡터 공간 모델

- Hinrich Schütze, 뮌헨대 (2014-04-30)

✍️ 메모

💡 이해 6강 핵심: 불리언의 한계 → 자카드 → tf(로그) → idf → tf-idf → 벡터 공간 + 코사인 유사도 → SMART 표기(lnc.ltn)

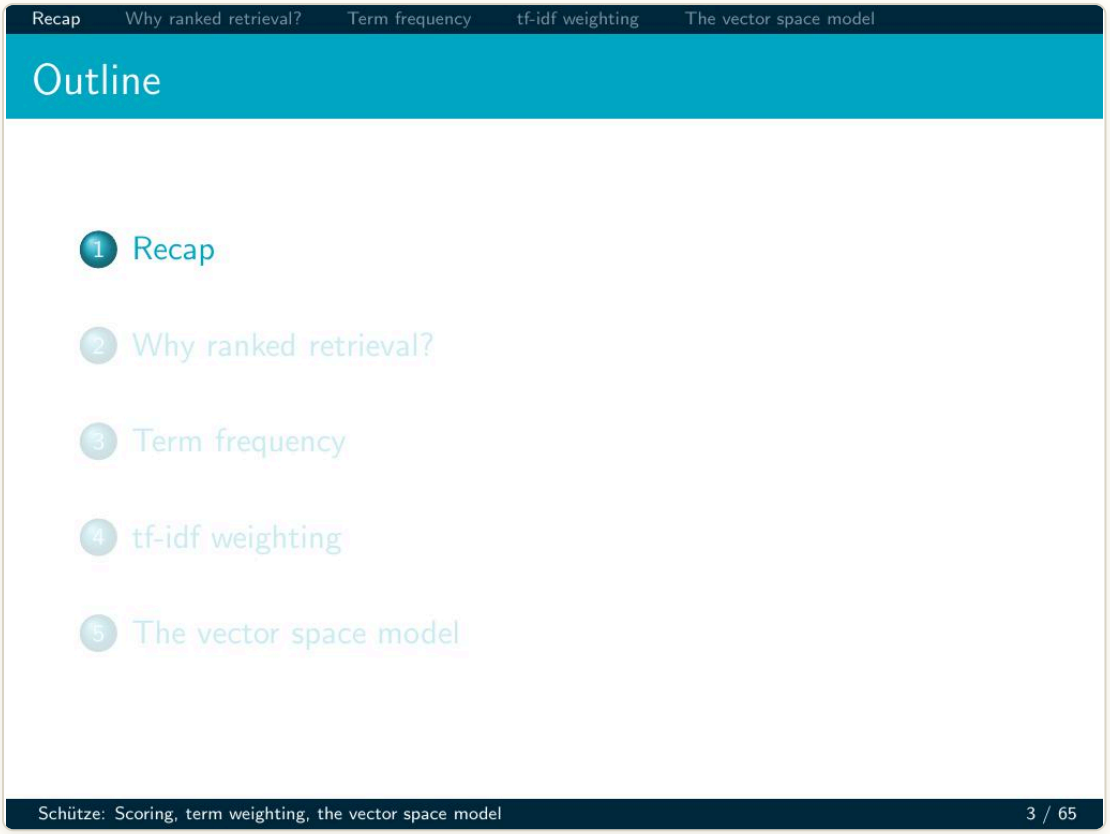


개요

- ① 복습 ② 왜 순위 검색인가? ③ 단어 빈도 ④ tf-idf 가중치 ⑤ 벡터 공간 모델

✍ 메모

— 넘어가도 되는 쪽



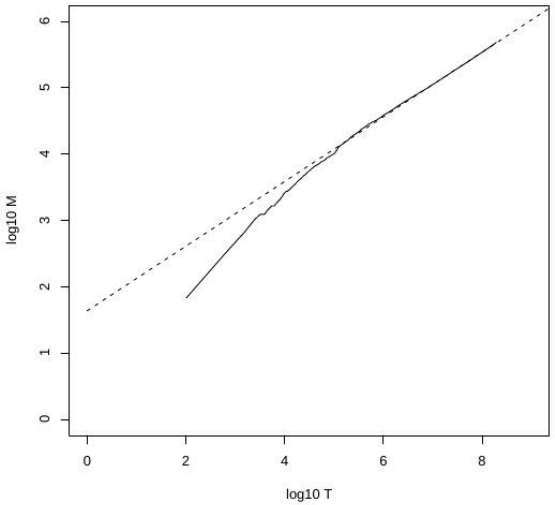
목차

- 복습 → 왜 순위 검색인가 → 단어 빈도 → tf-idf 가중치 → 벡터 공간 모델
- 지금: 복습

✍ 메모

— 넘어가도 되는 쪽

Heaps' law



Vocabulary size M as a function of collection size T (number of tokens) for Reuters-RCV1. For these data, the dashed line $\log_{10} M = 0.49 * \log_{10} T + 1.64$ is the best least squares fit. Thus, $M = 10^{1.64} T^{0.49}$ and $k = 10^{1.64} \approx 44$ and $b = 0.49$.

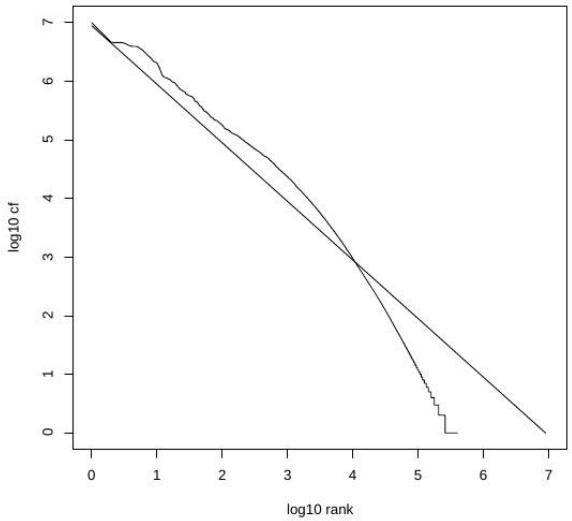
복습: 힙스의 법칙

- 로이터: $\log_{10}M = 0.49 \cdot \log_{10}T + 1.64 \rightarrow M = 10^{1.64} \cdot T^{0.49}$, $k \approx 44$, $b = 0.49$

✍ 메모

— 넘어가도 되는 쪽

Zipf's law



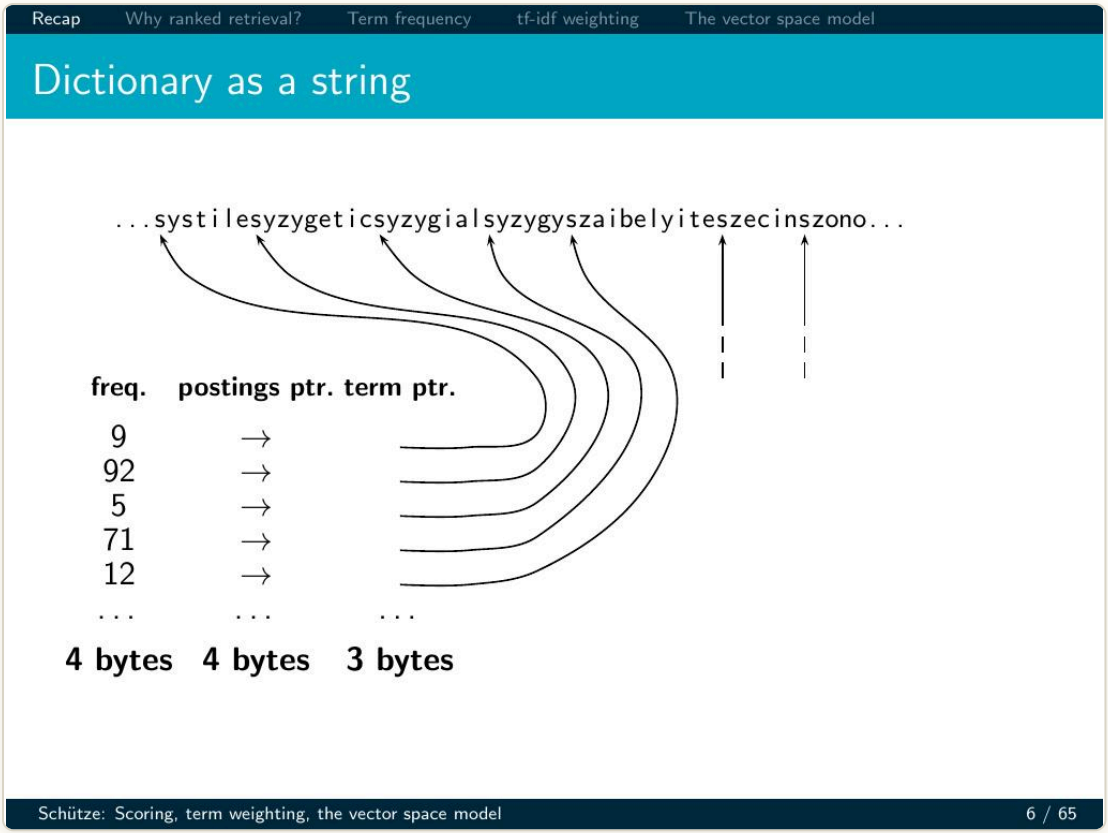
$\mathrm{cf}_i \propto \frac{1}{i}$
The most frequent term (*the*) occurs cf_1 times, the second most frequent term (*of*) occurs $\mathrm{cf}_2 = \frac{1}{2}\mathrm{cf}_1$ times, the third most frequent term (*and*) occurs $\mathrm{cf}_3 = \frac{1}{3}\mathrm{cf}_1$ times etc.

복습: 지프의 법칙

- $\mathrm{cf}_i \propto 1/i$ — the가 cf_1번이면 of는 ½cf_1, and는 ⅓cf_1 …

✍ 메모

— 넘어가도 되는 쪽

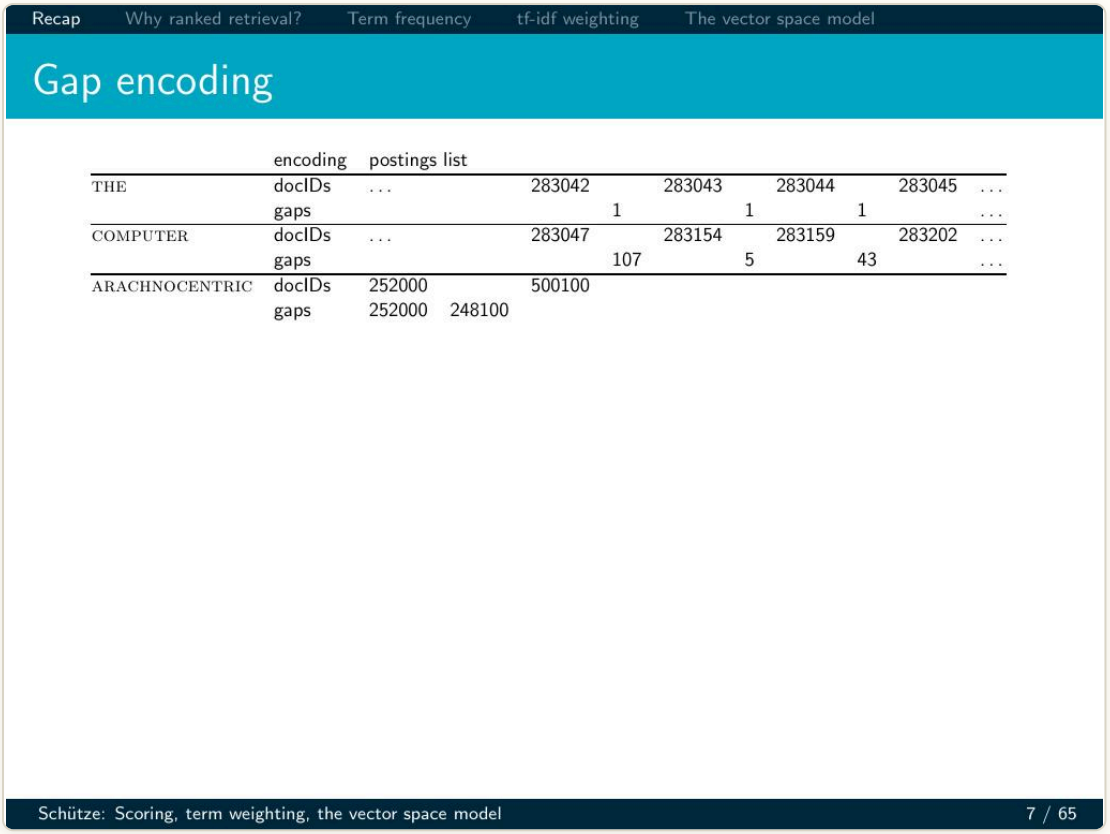


복습: 문자열로서의 사전

- ...systilesyzygeticsyzygialsyzygyszaibelyiteszecinszono... — 단어들을 이어 붙이고 위치 포인터로 접근

✍ 메모

— 넘어가도 되는 쪽

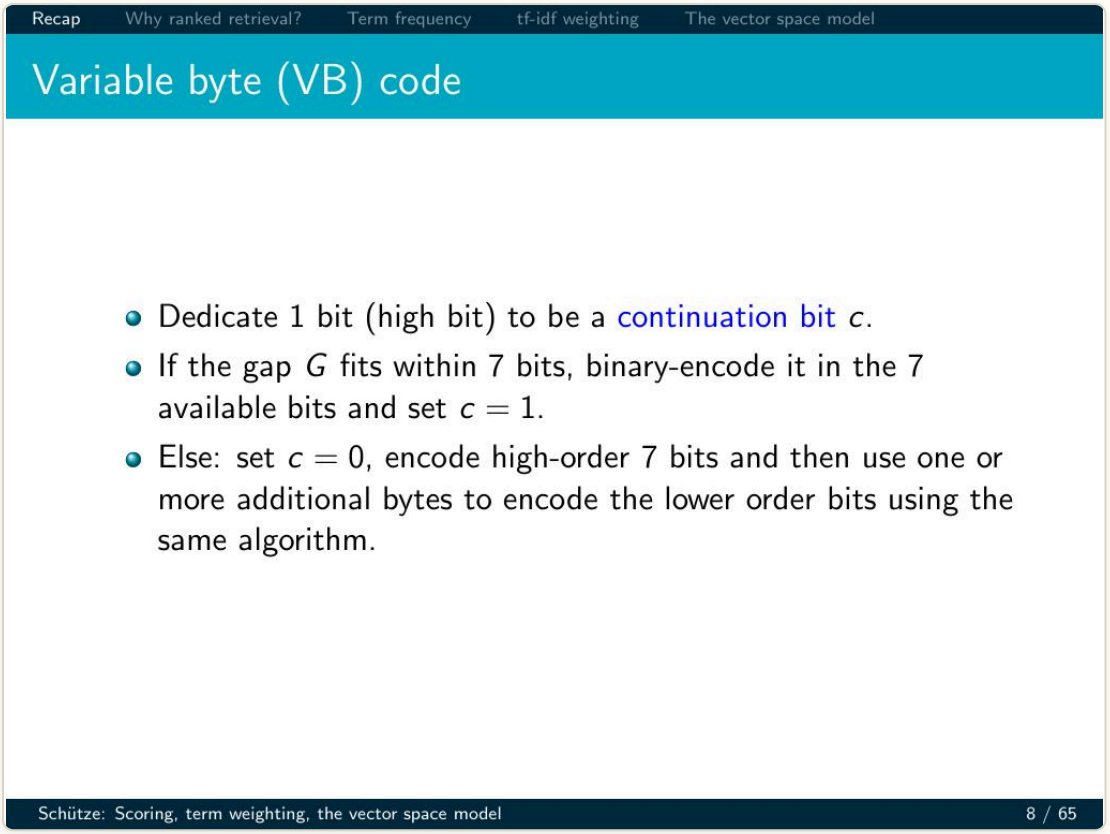


복습: 간격 인코딩

- THE: 간격 1, 1, 1 / COMPUTER: 107, 5, 43 / ARACHNOCENTRIC: 252000, 248100

✍ 메모

— 넘어가도 되는 쪽

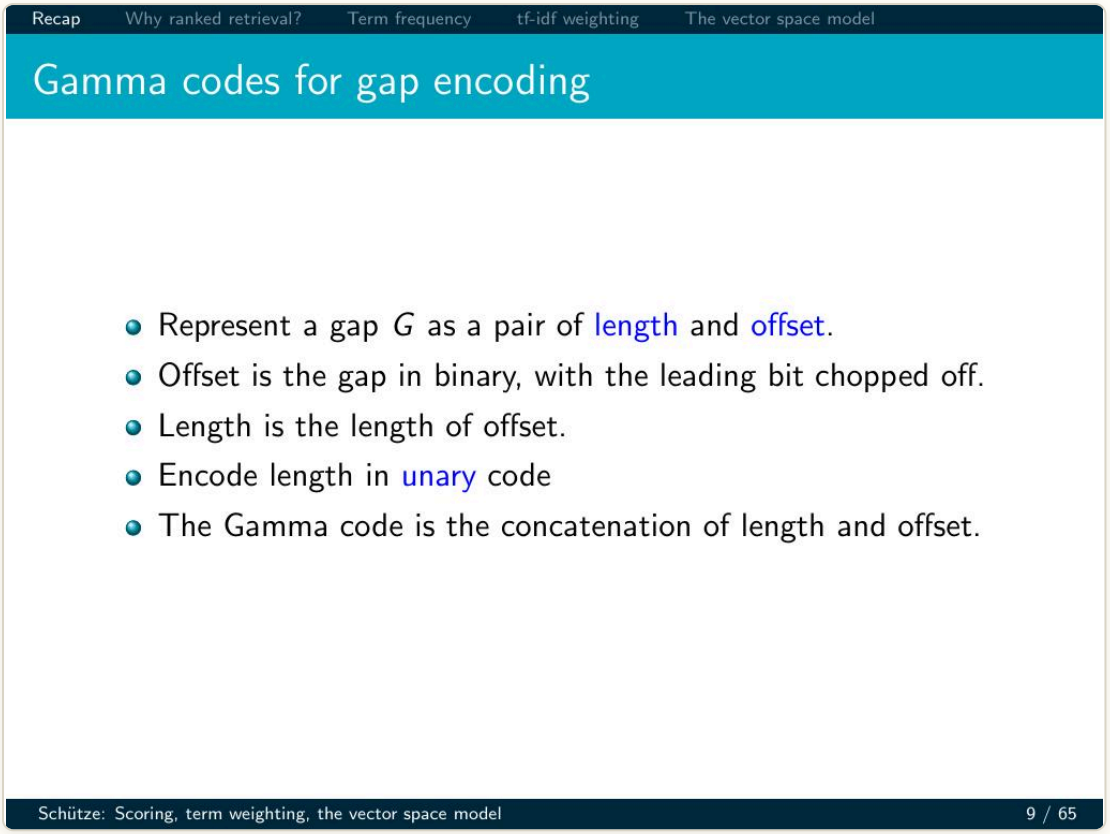


복습: 가변 바이트(VB) 코드

- 최상위 1비트 = 연속 비트 c
- 간격 G가 7비트에 들어가면 7비트에 인코딩하고 c = 1
- 아니면 c = 0으로 상위 7비트를 먼저 인코딩하고, 하위 비트는 같은 방식으로 추가 바이트에

✍ 메모

— 넘어가도 되는 쪽



복습: 감마 코드

- 간격 G = (길이, 오프셋) 쌍
- 오프셋 = 이진수에서 맨 앞 비트 제거, 길이 = 오프셋 길이를 단항 코드로
- 감마 코드 = 길이 + 오프셋

✍ 메모

— 넘어가도 되는 쪽

Compression of Reuters

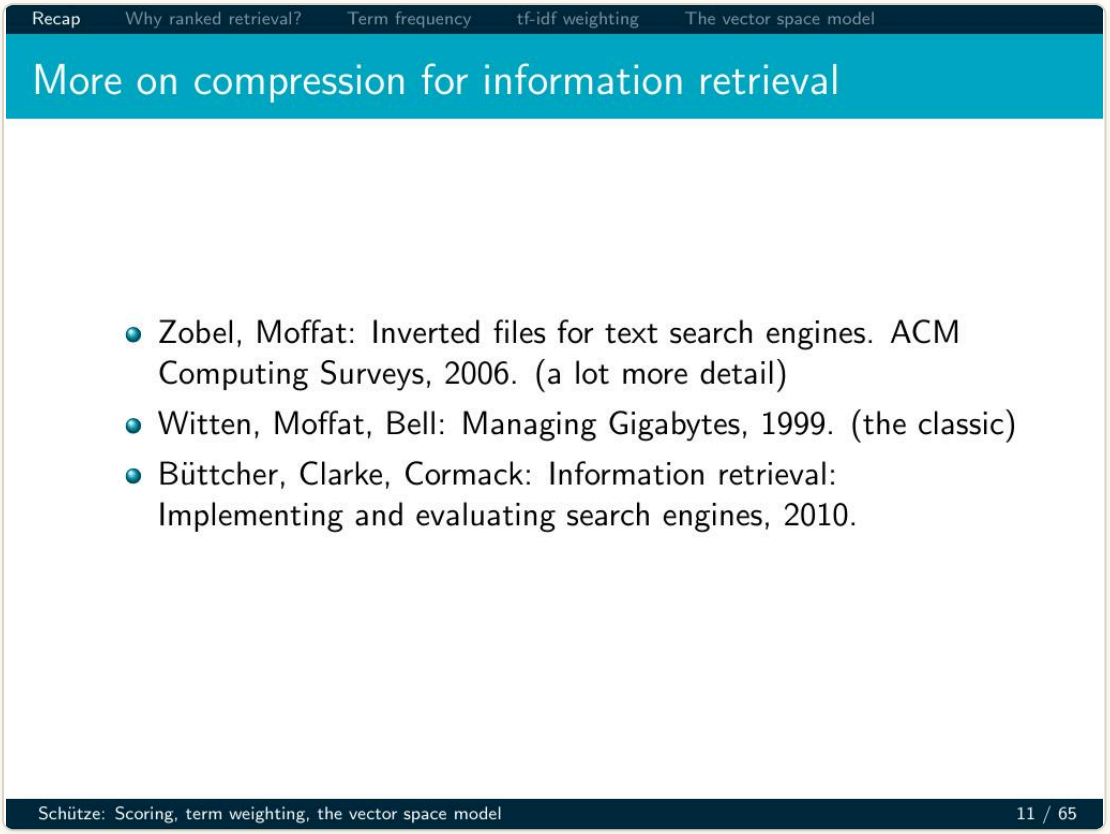
data structure	size in MB
dictionary, fixed-width	11.2
dictionary, term pointers into string	7.6
∼, with blocking, $k = 4$	7.1
∼, with blocking & front coding	5.9
collection (text, xml markup etc)	3600.0
collection (text)	960.0
T/D incidence matrix	40,000.0
postings, uncompressed (32-bit words)	400.0
postings, uncompressed (20 bits)	250.0
postings, variable byte encoded	116.0
postings, γ encoded	101.0

복습: 로이터 압축

- 사전 11.2 → 7.6 → 7.1 → 5.9MB / 포스팅 400 → 250 → VB 116 → γ 101MB

✍ 메모

— 넘어가도 되는 쪽

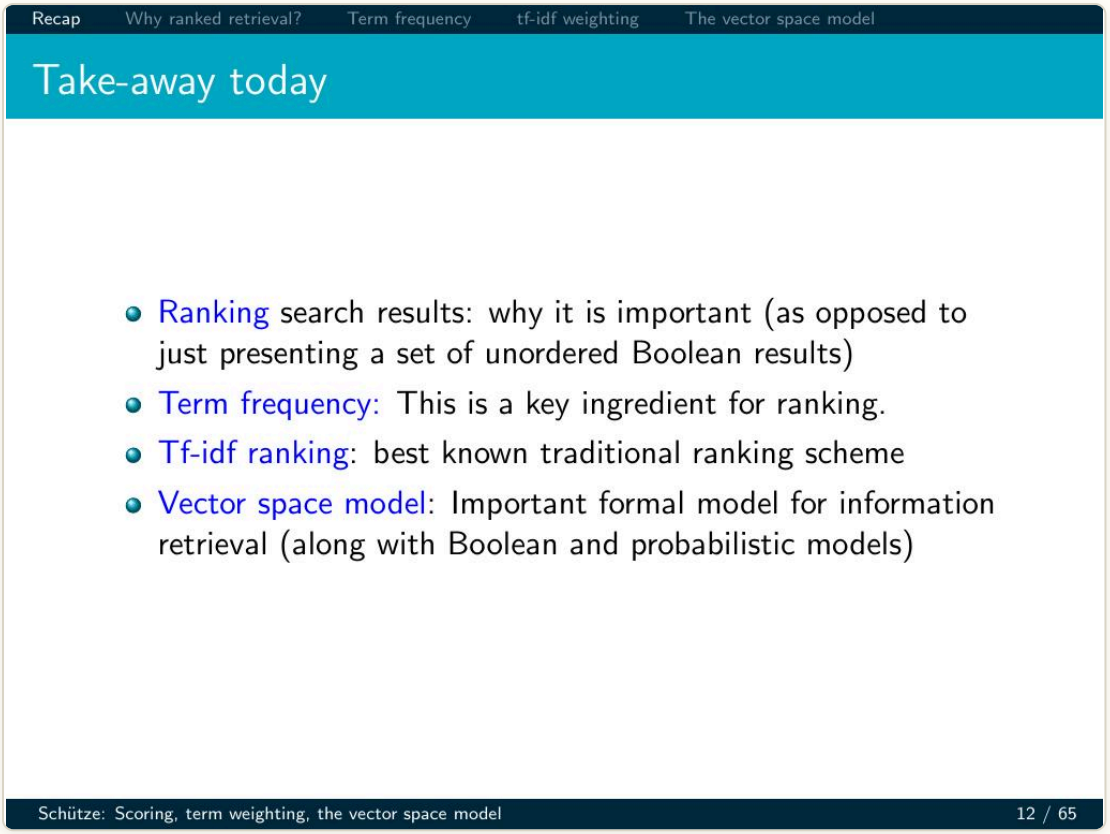


IR 압축에 대해 더 보기

- Zobel & Moffat: Inverted files for text search engines. ACM Computing Surveys, 2006 (훨씬 자세함)
- Witten, Moffat & Bell: Managing Gigabytes, 1999 (고전)
- Büttcher, Clarke & Cormack: Information retrieval: Implementing and evaluating search engines, 2010

✍ 메모

— 넘어가도 되는 쪽

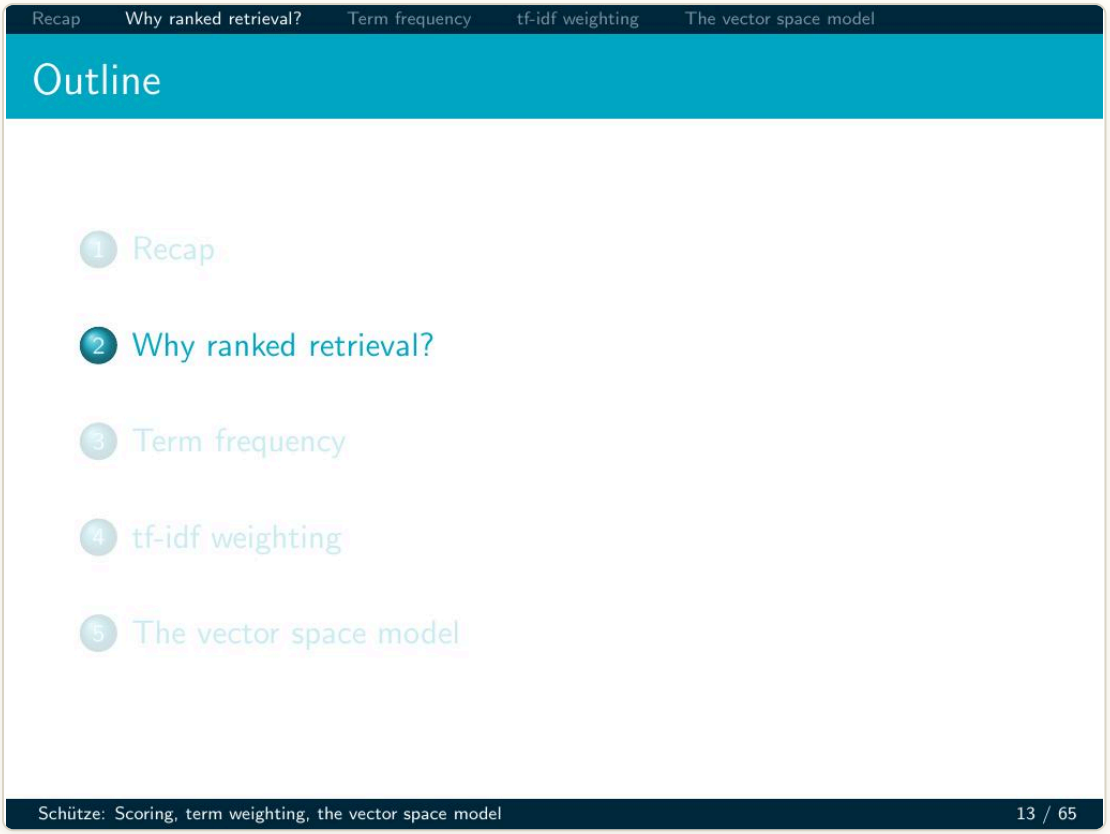


오늘 가져갈 것

- **검색 결과 순위화**: 왜 중요한가 (순서 없는 불리언 결과와 달리)
- **단어 빈도(tf)**: 순위화의 핵심 재료
- **tf-idf 순위화**: 가장 잘 알려진 전통적 순위 방식
- **벡터 공간 모델**: IR의 중요한 형식 모델 (불리언·확률 모델과 함께)

✍ 메모

— 넘어가도 되는 쪽

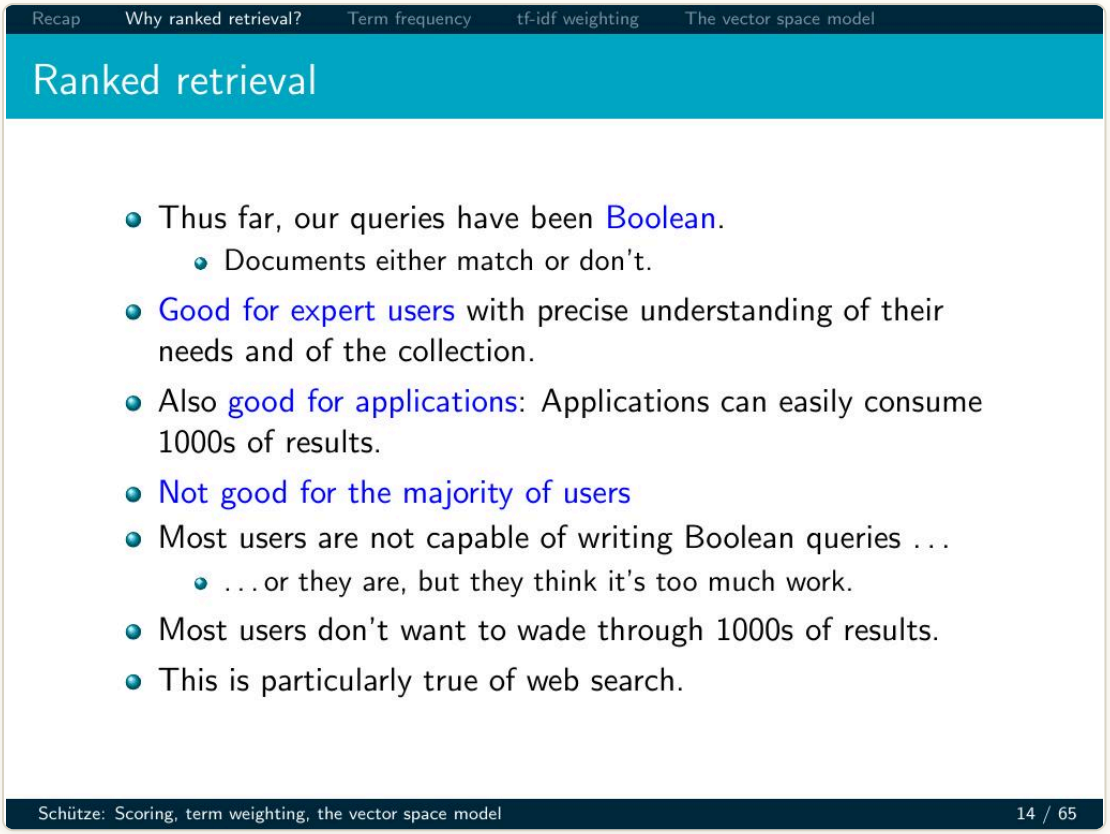


목차

- 복습 → 왜 순위 검색인가 → 단어 빈도 → tf-idf 가중치 → 벡터 공간 모델
- 지금: **왜 순위 검색인가?**

✍️ 메모

— 넘어가도 되는 쪽

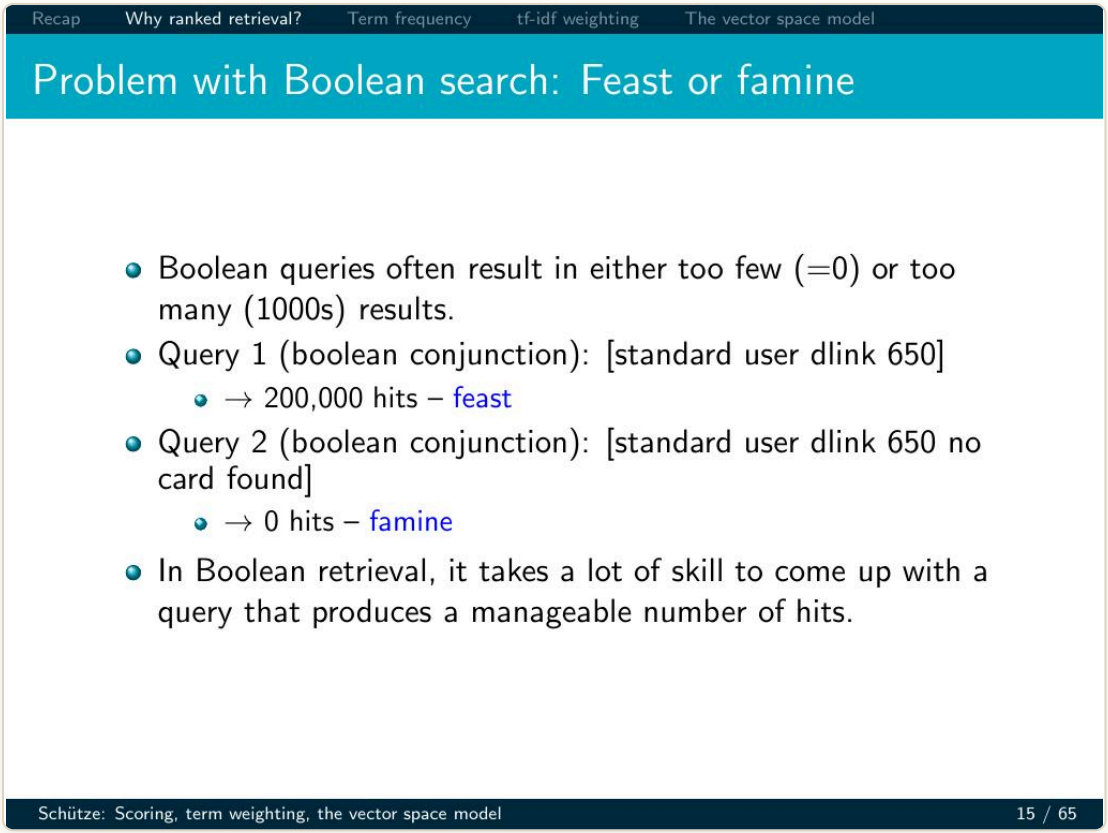


순위 검색 (Ranked retrieval)

- 지금까지 질의는 **불리언**이었다 — 문서는 맞거나 안 맞거나
- 자기 요구와 컬렉션을 정확히 아는 **전문 사용자**에겐 좋다
- **응용 프로그램**에도 좋다: 결과 수천 개도 쉽게 소화
- **대다수 사용자**에겐 좋지 않다
  - 대부분 불리언 질의를 쓸 줄 모른다 ...
  - ... 또는 쓸 줄 알아도 너무 귀찮다
  - 대부분 결과 수천 개를 헤치고 싶어 하지 않는다
  - **웹 검색**에서 특히 그렇다

✍ 메모

— 넘어가도 되는 쪽

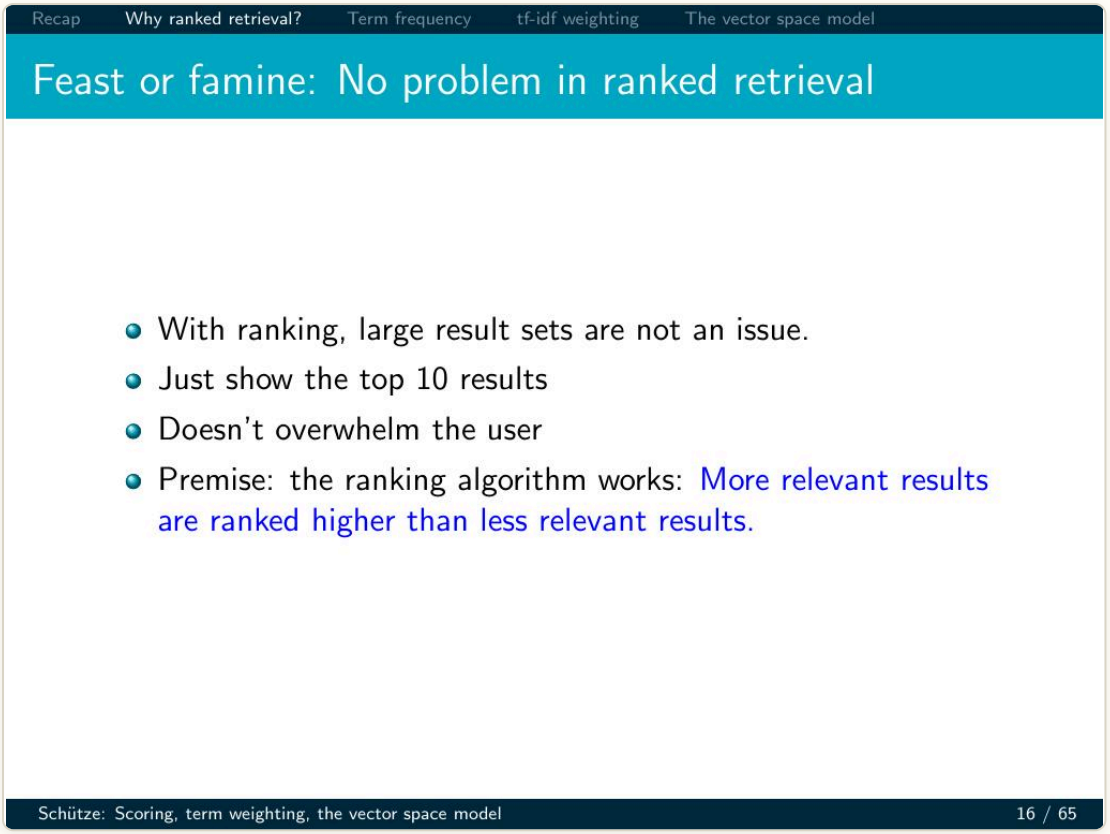


불리언 검색의 문제: 잔치 아니면 굶주림 (Feast or famine)

- 불리언 질의는 결과가 **너무 적거나(0)** **너무 많은(수천)** 경우가 많다
- 질의 1 (AND): [standard user dlink 650] → **200,000건 — 잔치**
- 질의 2 (AND): [standard user dlink 650 no card found] → **0건 — 굶주림**
- 불리언 검색에서 다룰 만한 결과 수가 나오는 질의를 만들려면 **상당한 기술**이 필요

✍ 메모

★ 시험 **feast or famine = 불리언의 대표 단점**

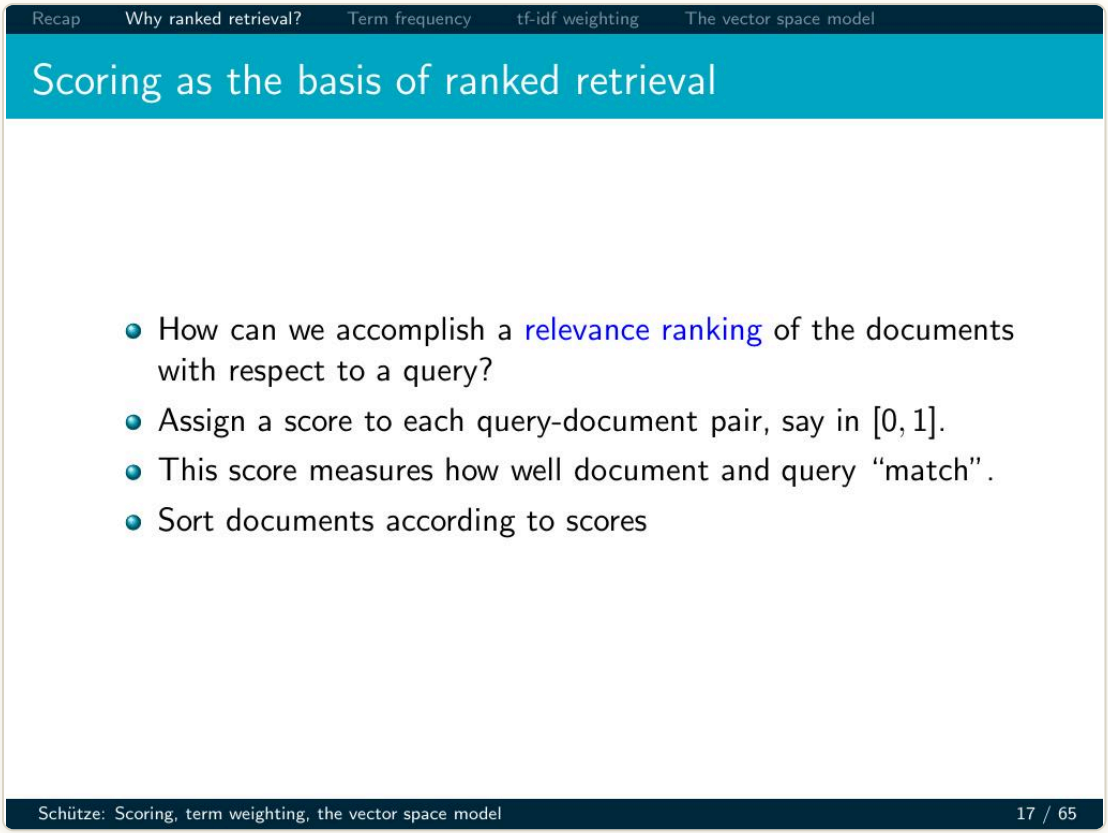


순위 검색에선 문제가 안 된다

- 순위가 있으면 결과 집합이 커도 문제없다
  - **상위 10개**만 보여 주면 된다
  - 사용자를 압도하지 않는다
- 전제: 순위 알고리즘이 잘 동작한다 — 더 관련 있는 결과가 더 위에

✍ 메모

— 넘어가도 되는 쪽

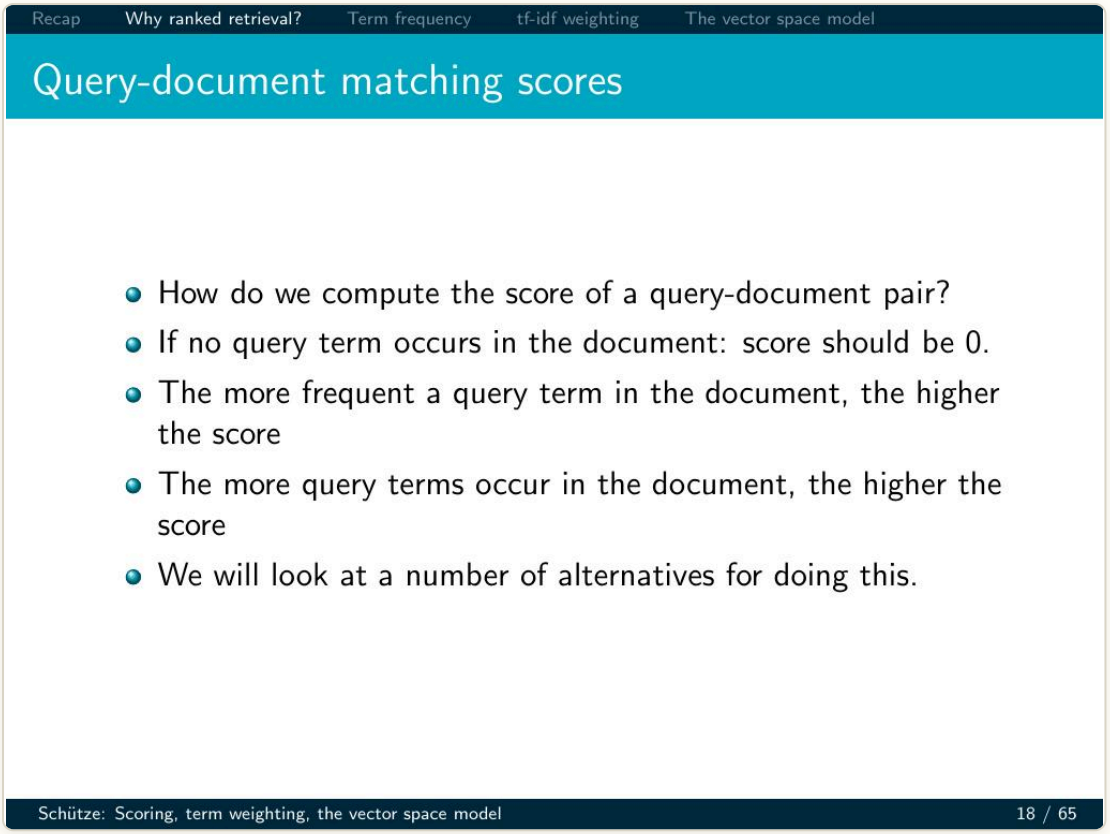


순위 검색의 기반인 점수화

- 질의에 대해 문서의 **관련성 순위**를 어떻게 매길까?
- 각 **질의-문서 쌍에 점수**를 준다, 예: [0, 1]
- 이 점수는 문서와 질의가 얼마나 잘 '맞는지'를 잰다
- 점수로 문서를 정렬

✍ 메모

— 넘어가도 되는 쪽

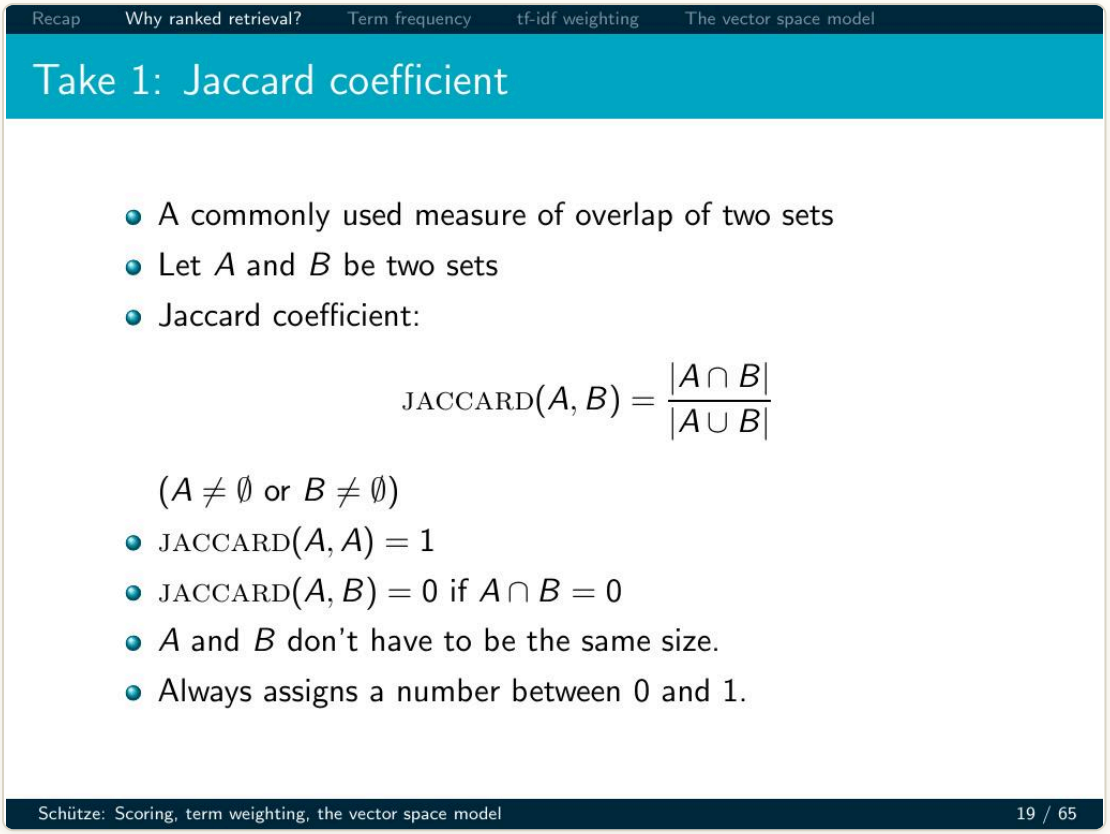


질의-문서 일치 점수

- 질의-문서 쌍의 점수를 어떻게 계산하나?
- 질의어가 문서에 **하나도 없으면** 점수는 **0**
- 질의어가 문서에 **자주** 나올수록 점수↑
- 질의어가 문서에 **많이** 나올수록 점수↑
- 여러 대안을 살펴본다

✍ 메모

— 넘어가도 되는 쪽

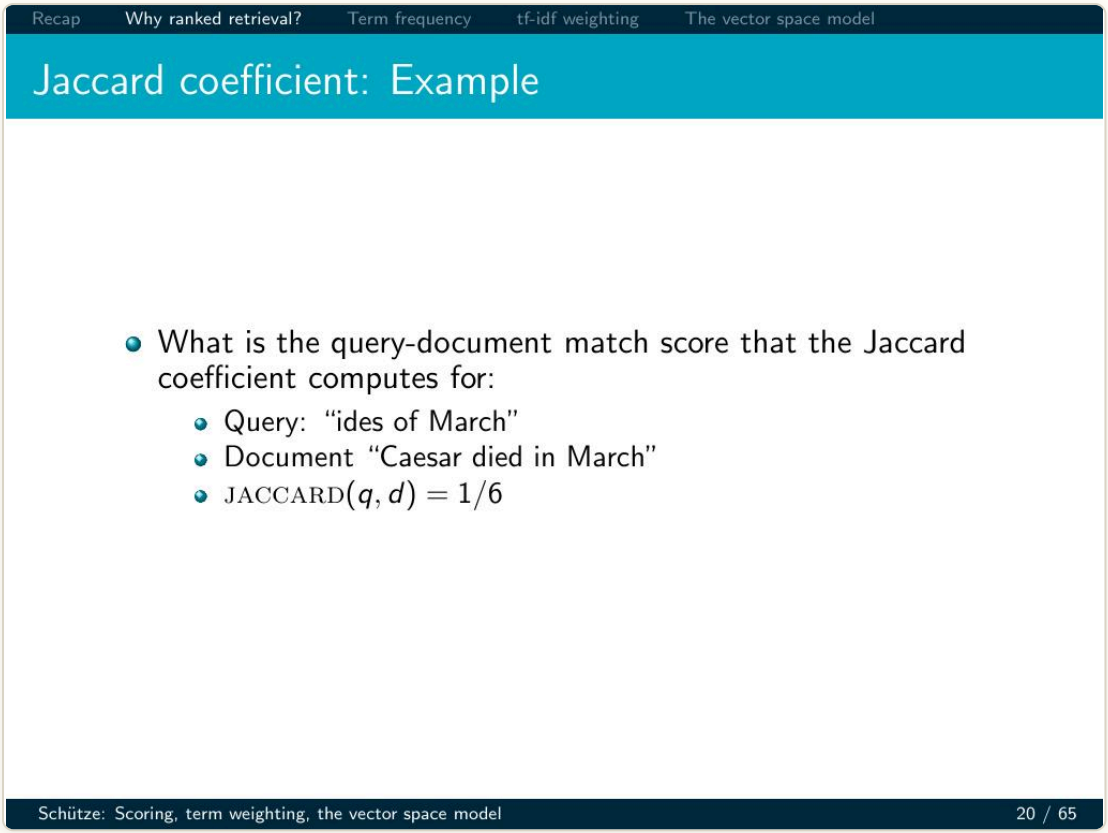


시도 1: 자카드 계수 (Jaccard coefficient)

- 두 집합의 **겹침**을 재는 흔한 척도
- A, B를 두 집합이라 하자
- **JACCARD(A, B) = |A ∩ B| / |A ∪ B|** (A ≠ ∅ 또는 B ≠ ∅)
- JACCARD(A, A) = 1
- A ∩ B = ∅이면 JACCARD(A, B) = 0
- A와 B의 **크기가 같을 필요 없다**
- 항상 **0과 1 사이**의 값

✍️ 메모

★ 시험 자카드 = 교집합 / 합집합

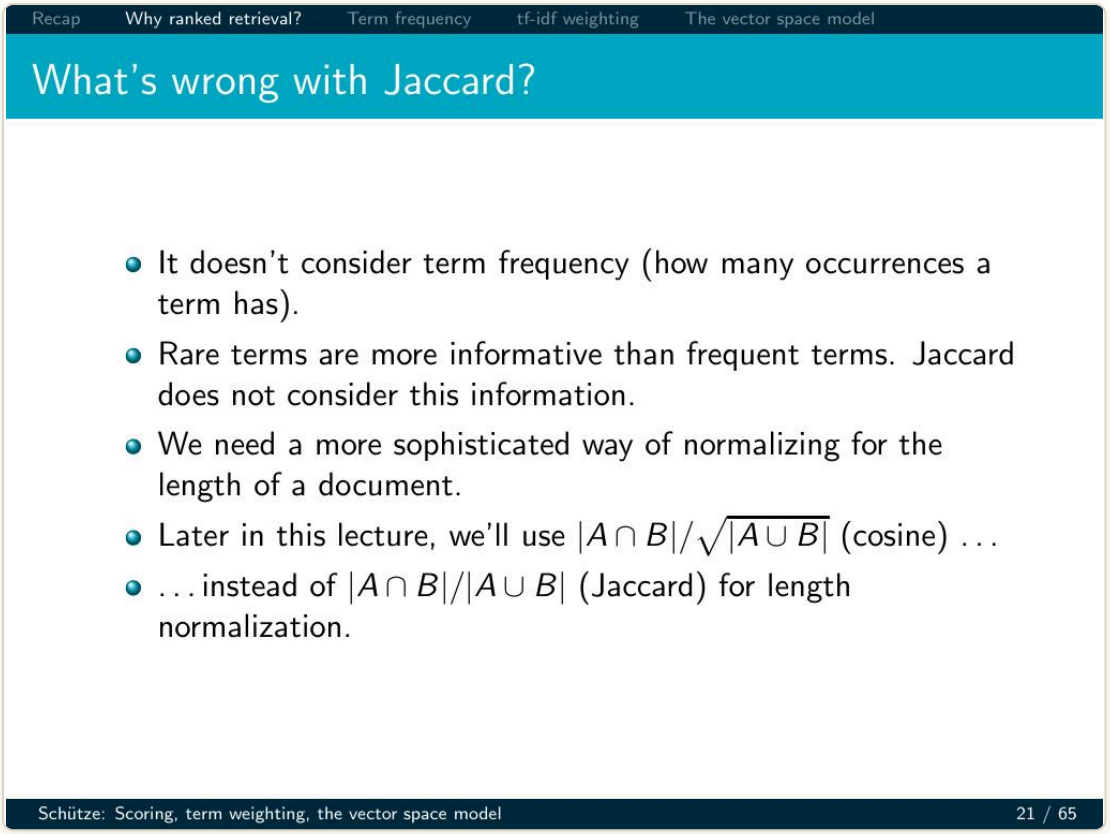


자카드 계수: 예

- 질의 "ides of March"와 문서 "Caesar died in March"의 자카드 일치 점수는?
- **jaccard(q, d) = 1/6**

✍ 메모

✏ 예·풀이 ∩ = {march} 1개, ∪ = {ides, of, march, caesar, died, in} 6개 → 1/6

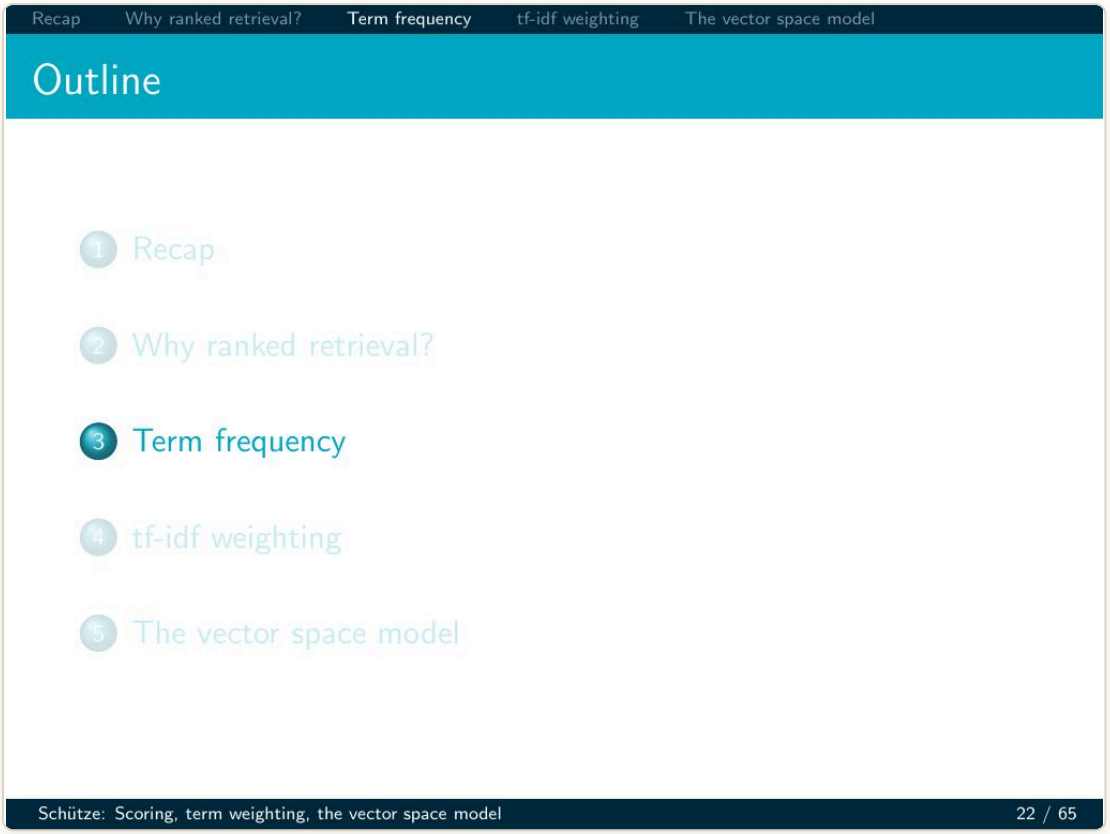


자카드의 문제

- **단어 빈도**(몇 번 나오나)를 고려하지 않는다
- **드문 단어가 흔한 단어보다 정보량이 많은데**, 자카드는 이 정보를 쓰지 않는다
- 문서 **길이 정규화**를 더 정교하게 해야
- 이 강의 뒤에서 |A ∩ B| / |A ∪ B| (자카드) 대신 |A ∩ B| / √|A ∪ B| (코사인)를 길이 정규화에 쓴다

✍ 메모

★ 시험 자카드 문제 3개: tf 무시 · 희소성(idf) 무시 · 길이 정규화 부실



목차

- 복습 → 왜 순위 검색인가 → 단어 빈도 → tf-idf 가중치 → 벡터 공간 모델
- 지금: **단어 빈도**

✍ 메모

— 넘어가도 되는 쪽

Binary incidence matrix

	Anthony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	...
ANTHONY	1	1	0	0	0	1	
BRUTUS	1	1	0	1	0	0	
CAESAR	1	1	0	1	1	1	
CALPURNIA	0	1	0	0	0	0	
CLEOPATRA	1	0	0	0	0	0	
MERCY	1	0	1	1	1	1	
WORSER	1	0	1	1	1	0	
...							

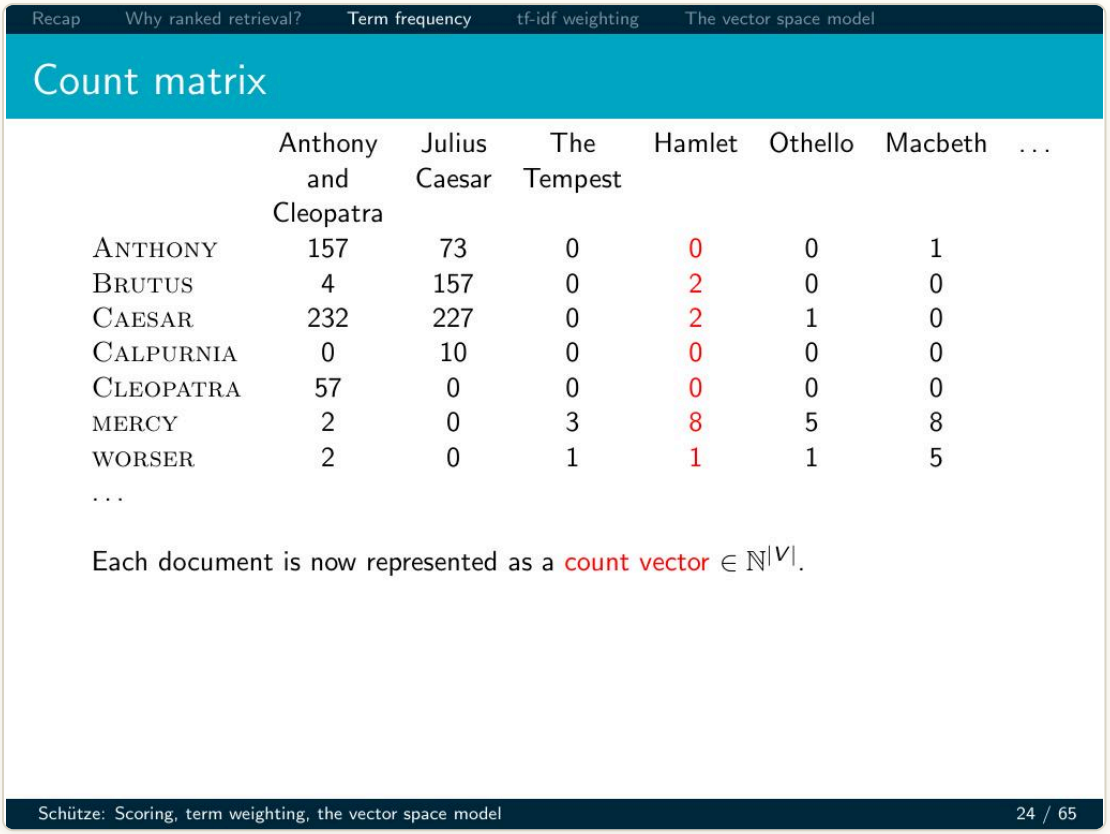
Each document is represented as a binary vector $\in \{0,1\}^{|V|}$.

이진 발생 행렬

- 1강의 0/1 행렬
- 각 문서를 **이진 벡터** ∈ {0, 1}^|V|로 표현

✍ 메모

— 넘어가도 되는 쪽

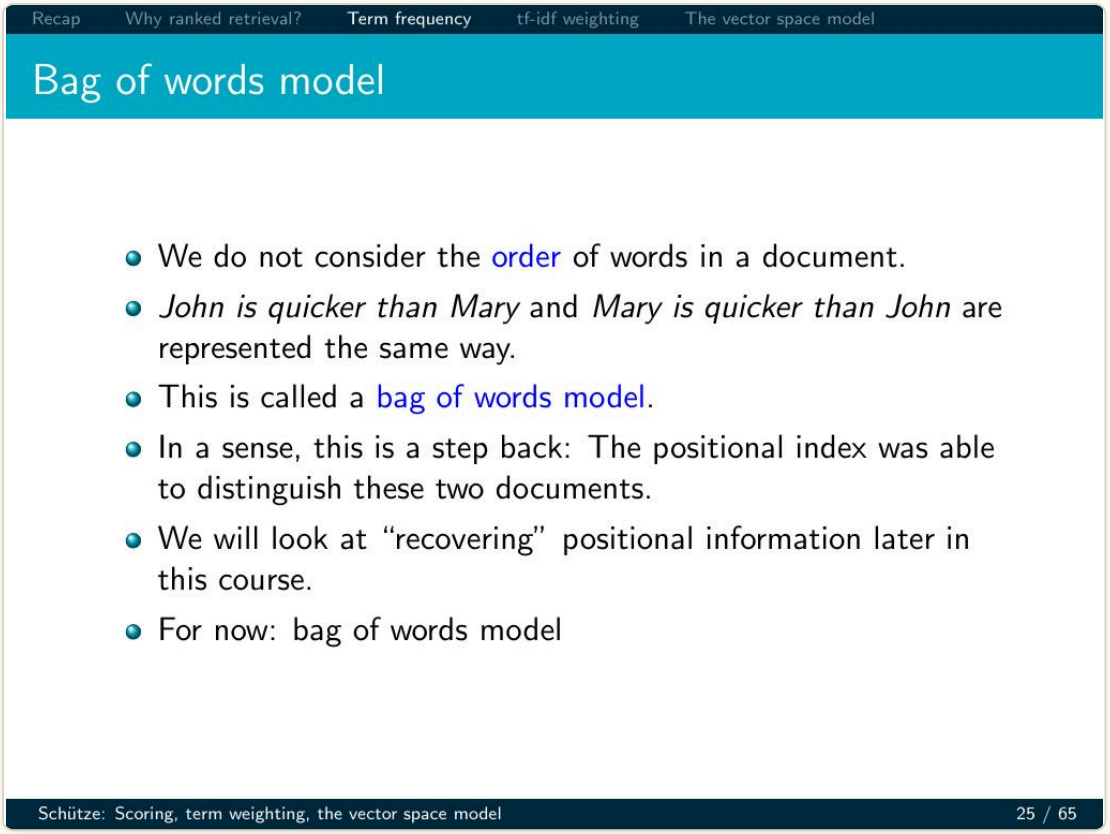


개수 행렬 (Count matrix)

- 행 = 단어, 열 = 희곡, 칸 = 등장 횟수
  - Anthony 157 73 0 0 0 1 / Brutus 4 157 0 2 0 0 / Caesar 232 227 0 2 1 0 / Calpurnia 0 10 0 0 0 0
  - Cleopatra 57 0 0 0 0 0 / mercy 2 0 3 8 5 8 / worser 2 0 1 1 1 5
- 각 문서가 이제 **개수 벡터** ∈ ℕ^|V|

✍ 메모

— 넘어가도 되는 쪽

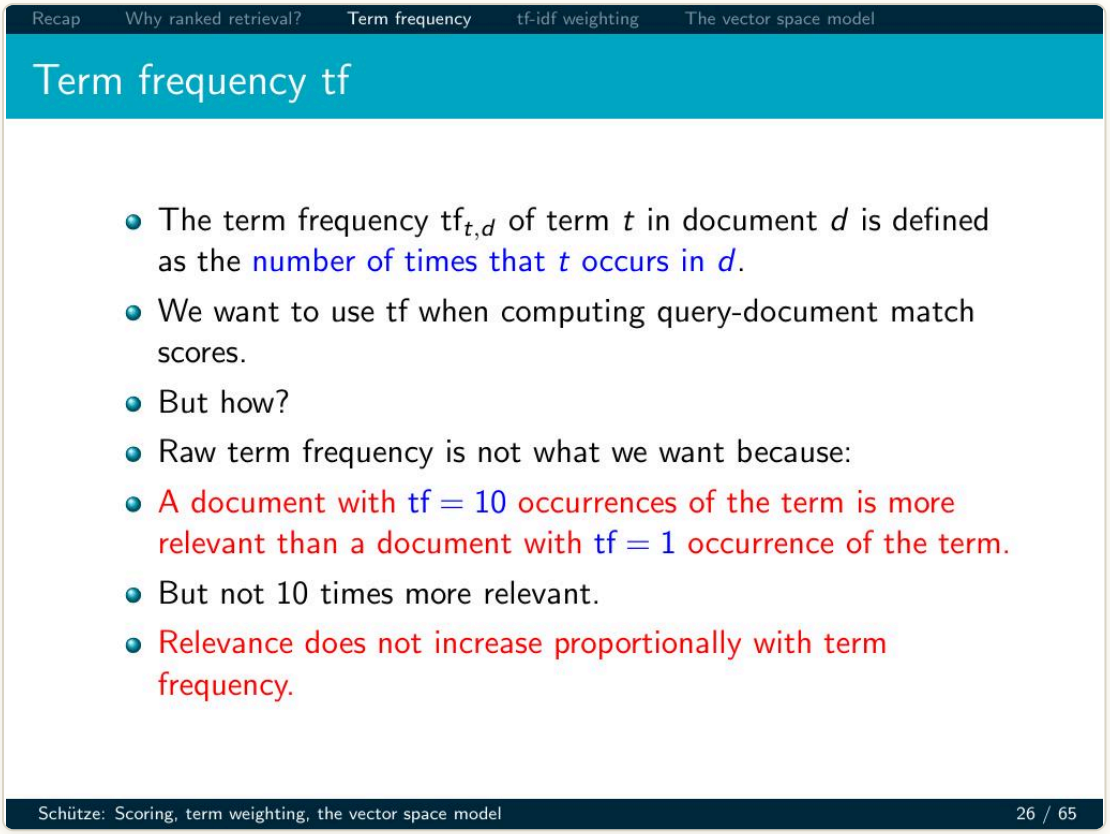


단어 가방 모델 (Bag of words)

- 문서 안 단어의 **순서는 고려하지 않는다**
- John is quicker than Mary와 Mary is quicker than John이 **같게** 표현된다
- 이를 **단어 가방 모델**이라 한다
- 어떤 의미로는 후퇴: 위치 색인은 두 문서를 구분할 수 있었다
- 위치 정보를 '되찾는' 방법은 나중에
- 지금은 단어 가방 모델

✍ 메모

★ 시험 **bag of words = 순서 무시, 빈도만**

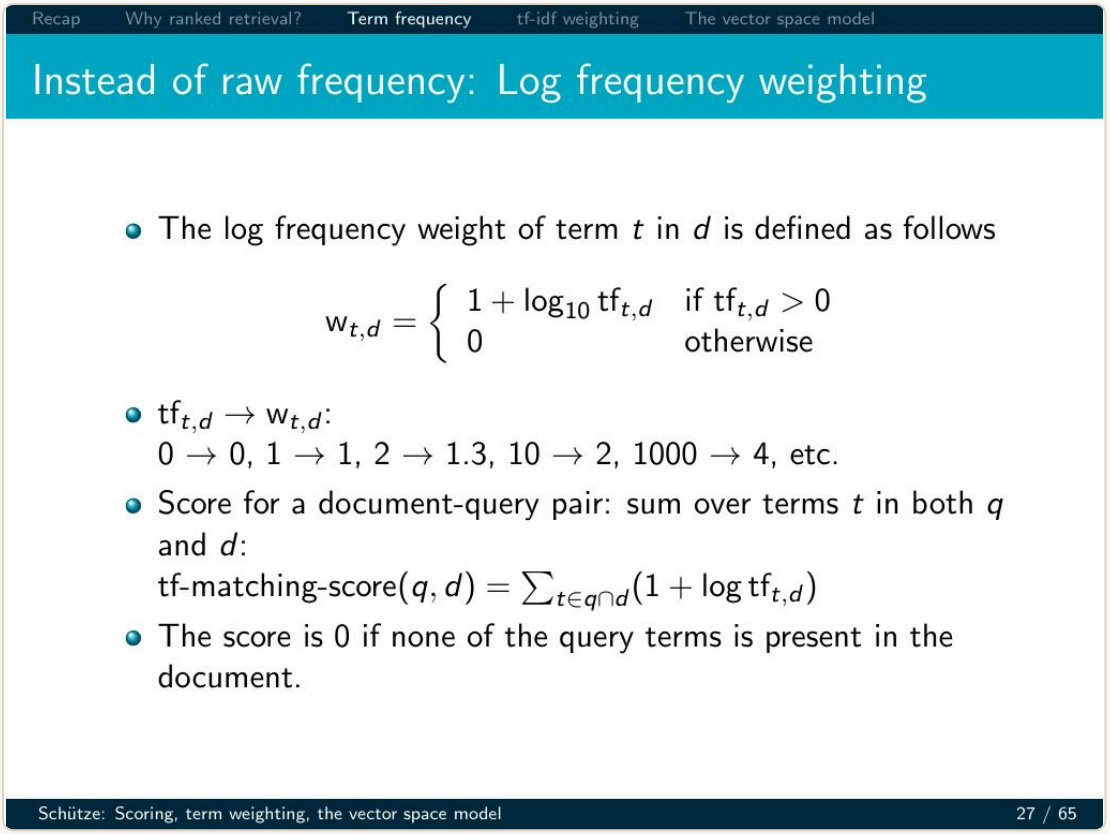


단어 빈도 tf

- 문서 d에서 단어 t의 **단어 빈도 tf(t,d)** = d에 t가 나온 횟수
- 질의-문서 일치 점수 계산에 tf를 쓰고 싶다
- 그런데 어떻게?
- **원시 단어 빈도**는 원하는 게 아니다:
  - tf = 10인 문서가 tf = 1인 문서보다 더 관련 있지만
  - **10배 더 관련 있는 건 아니다**
- 관련성은 단어 빈도에 **비례해서** 늘지 않는다

✍ 메모

— 넘어가도 되는 쪽

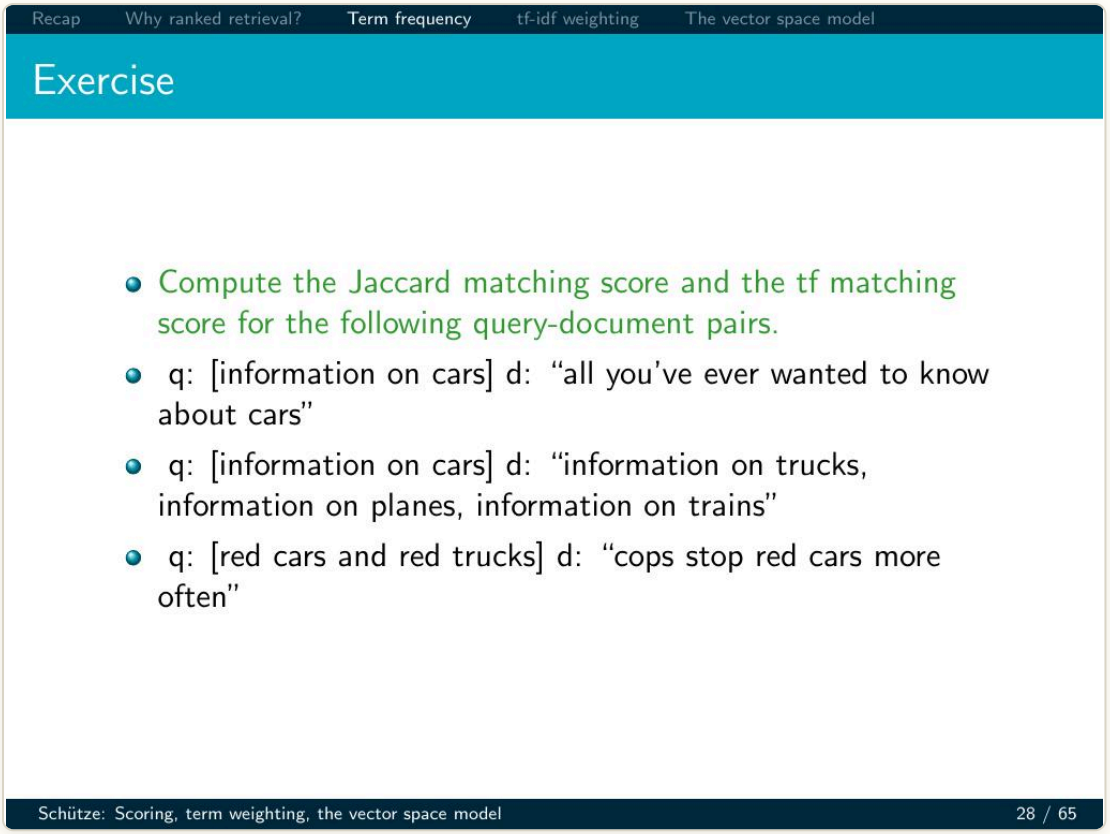


원시 빈도 대신: 로그 빈도 가중치

- 문서 d에서 단어 t의 **로그 빈도 가중치**
  - **w(t,d) = 1 + log₁₀ tf(t,d)** (tf > 0일 때), 아니면 **0**
- tf → w: 0 → 0, 1 → 1, 2 → 1.3, 10 → 2, 1000 → 4 등
- 문서-질의 쌍의 점수: q와 d **둘 다에 있는** 단어 t에 대해 합
  - **tf-matching-score(q, d) = Σ_{t ∈ q∩d} (1 + log tf(t,d))**
- 질의어가 문서에 하나도 없으면 점수는 0

✍ 메모

★ 시험 1 + log₁₀tf: 1→1, 2→1.3, 10→2, 100→3, 1000→4



연습

- 다음 질의-문서 쌍의 **자카드 점수**와 **tf 점수**를 구하라
- ① q: [information on cars] d: "all you've ever wanted to know about cars"
- ② q: [information on cars] d: "information on trucks, information on planes, information on trains"
- ③ q: [red cars and red trucks] d: "cops stop red cars more often"

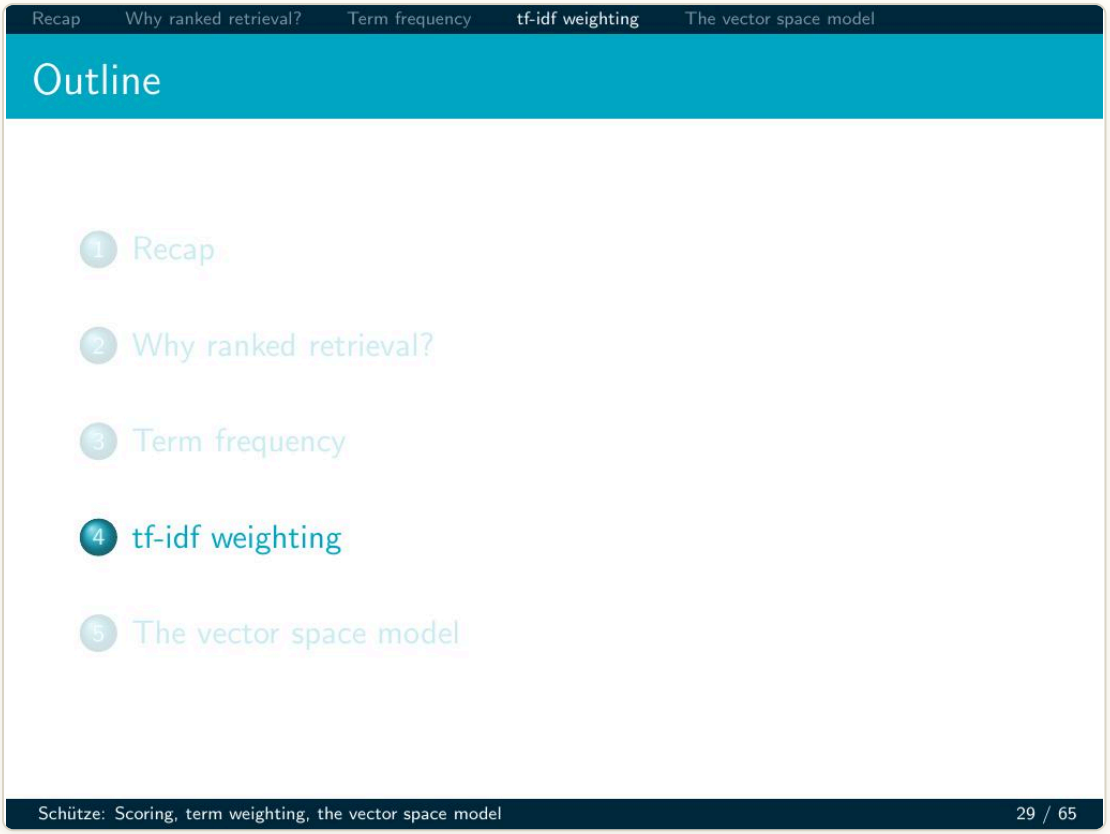
✍ 메모

✏ 예·풀이 ① ∩{cars}=1, ∪=10 → 자카드 1/10 / tf: cars 1회 → 1

✏ 예·풀이 ② ∩{information, on}=2, ∪=6 → 1/3 / tf: 각 3회 → (1+log3)×2 ≈ 2.95

✏ 예·풀이 ③ ∩{red, cars}=2, ∪=8 → 1/4 / tf: red 1, cars 1 → 2

💡 이해 ②는 자카드·tf 모두 높지만 실제론 cars가 없어 관련 없다 → idf가 필요한 이유

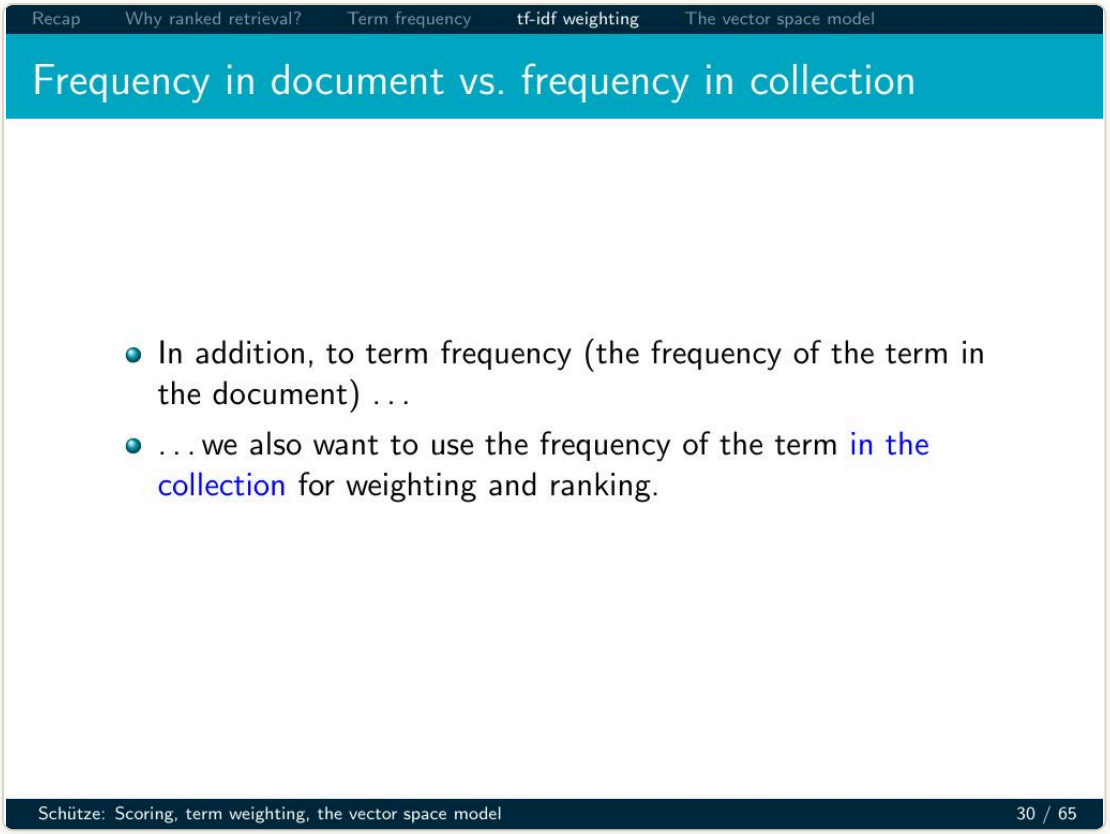


목차

- 복습 → 왜 순위 검색인가 → 단어 빈도 → tf-idf 가중치 → 벡터 공간 모델
- 지금: **tf-idf 가중치**

✍ 메모

— 넘어가도 되는 쪽

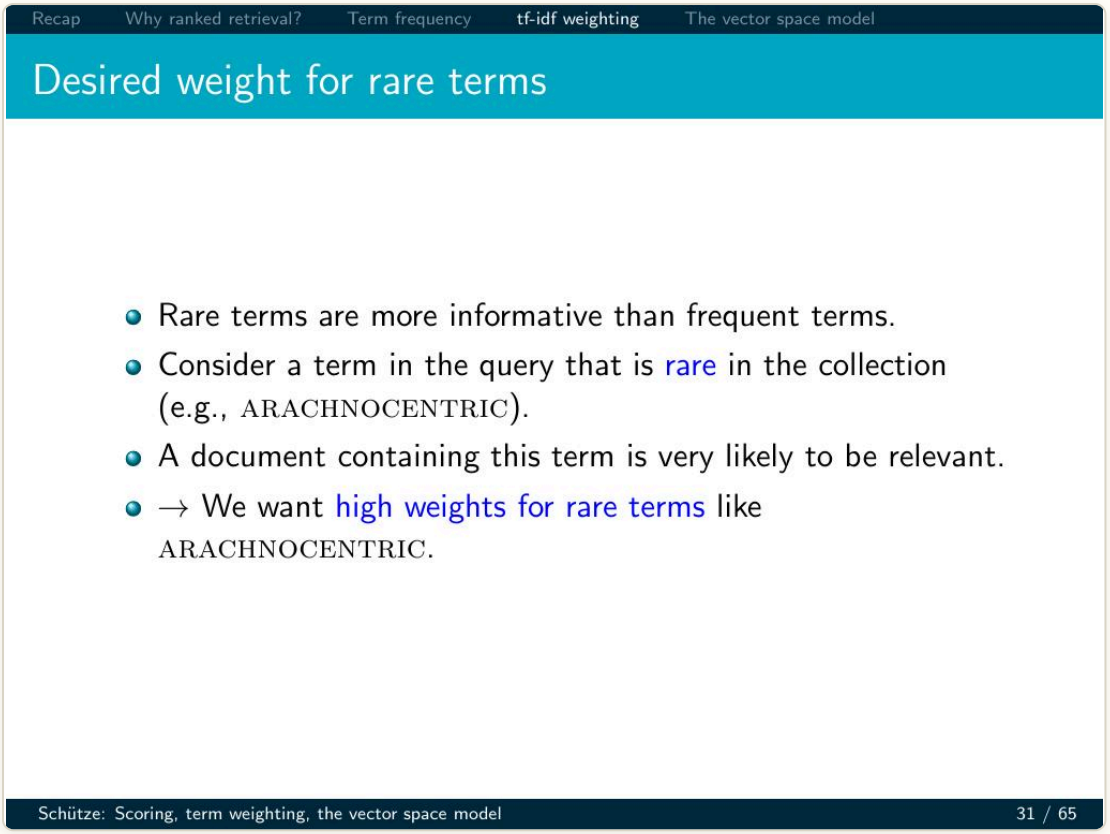


문서 안 빈도 vs 컬렉션 안 빈도

- **단어 빈도**(문서 안에서의 빈도)에 더해 ...
- ... 가중치와 순위에 **컬렉션에서의 빈도**도 쓰고 싶다

✍ 메모

— 넘어가도 되는 쪽

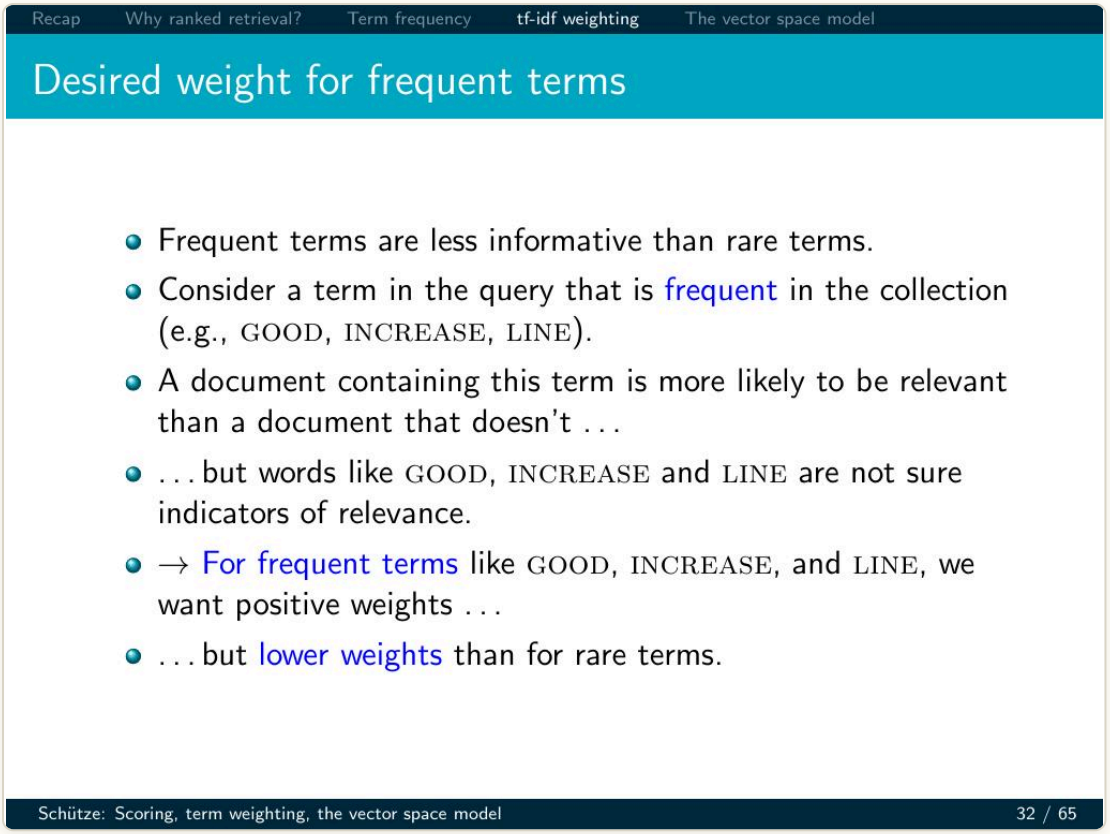


드문 단어에 원하는 가중치

- **드문 단어가 흔한 단어보다 정보량이 많다**
- 컬렉션에서 드문 질의어를 생각해 보자 (예: arachnocentric)
- 이 단어가 있는 문서는 관련 있을 가능성이 **매우 높다**
- → arachnocentric 같은 드문 단어엔 **높은 가중치**

✍ 메모

— 넘어가도 되는 쪽

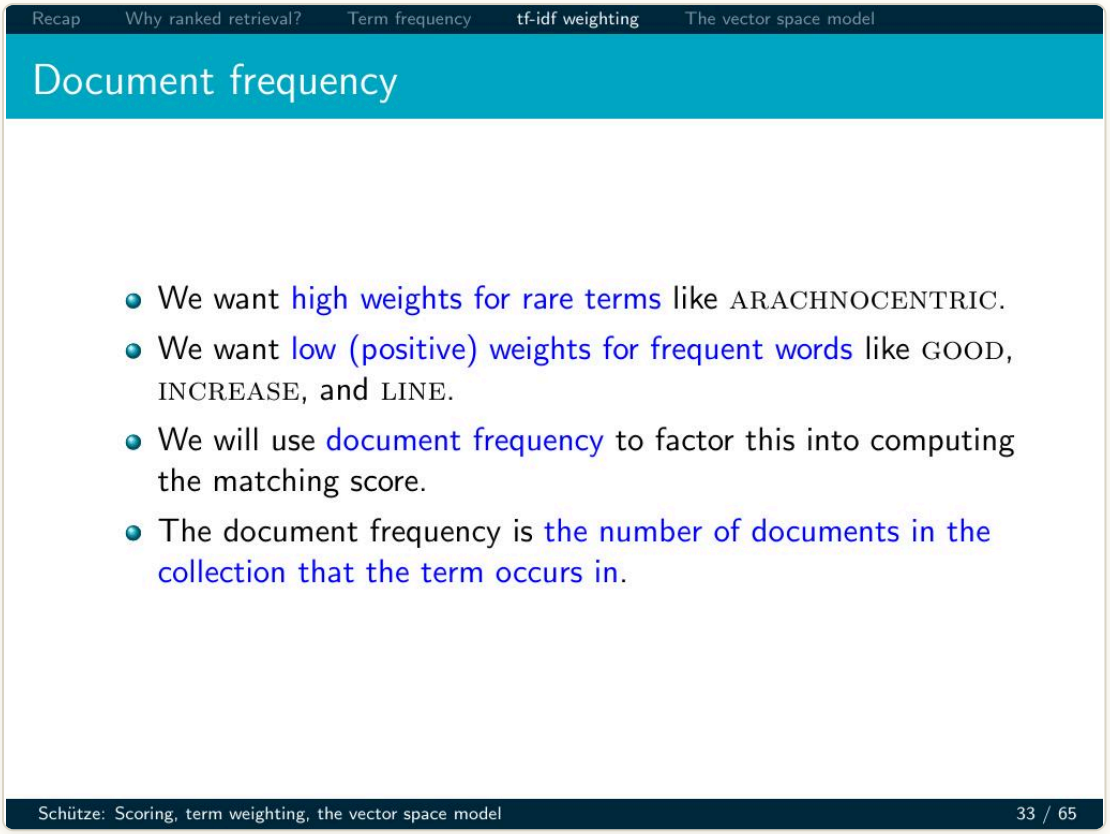


흔한 단어에 원하는 가중치

- 흔한 단어는 드문 단어보다 정보량이 적다
- 컬렉션에서 흔한 질의어를 생각해 보자 (예: good, increase, line)
- 이 단어가 있는 문서가 없는 문서보다 관련 있을 가능성은 크지만 ...
- ... good, increase, line 같은 단어는 관련성의 **확실한 지표가 아니다**
- → 흔한 단어엔 **양의 가중치**를 주되 ...
- ... 드문 단어보다는 **낮게**

✍ 메모

— 넘어가도 되는 쪽



문서 빈도 (Document frequency)

- arachnocentric 같은 드문 단어엔 높은 가중치
- good, increase, line 같은 흔한 단어엔 낮은 (양의) 가중치
- 이를 일치 점수에 반영하려고 **문서 빈도**를 쓴다
- 문서 빈도 = 컬렉션에서 그 단어가 나오는 **문서의 수**

✍ 메모

— 넘어가도 되는 쪽

Recap Why ranked retrieval? Term frequency tf-idf weighting The vector space model

idf weight

- df_t is the document frequency, the number of documents that t occurs in.
- df_t is an inverse measure of the informativeness of term t.
- We define the idf weight of term t as follows:

$$idf_t = \log_{10} \frac{N}{df_t}$$

(N is the number of documents in the collection.)

- idf_t is a measure of the informativeness of the term.
- $[\log N/df_t]$ instead of $[N/df_t]$ to "dampen" the effect of idf
- Note that we use the log transformation for both term frequency and document frequency.

Schütze: Scoring, term weighting, the vector space model 34 / 65

idf 가중치

- df(t) = 단어 t가 나오는 문서 수
- df(t)는 단어 t의 정보량에 대한 **역** 척도
- 단어 t의 **idf 가중치**: **idf(t) = $\log_{10}$ (N / df(t))**
  - (N = 컬렉션의 문서 수)
- idf(t)는 단어의 **정보량** 척도
- idf 효과를 '누그러뜨리려고' N/df 대신 **log** N/df
- **tf와 df 둘 다 로그 변환**을 쓴다는 점에 주의

✍ 메모

★ 시험 idf = $\log_{10}$(N/df) — 드물수록 크다

Examples for idf

Compute idf_t using the formula: $\text{idf}_t = \log_{10} \frac{1{,}000{,}000}{\text{df}_t}$

term	df_t	idf_t
calpurnia	1	6
animal	100	4
sunday	1000	3
fly	10,000	2
under	100,000	1
the	1,000,000	0

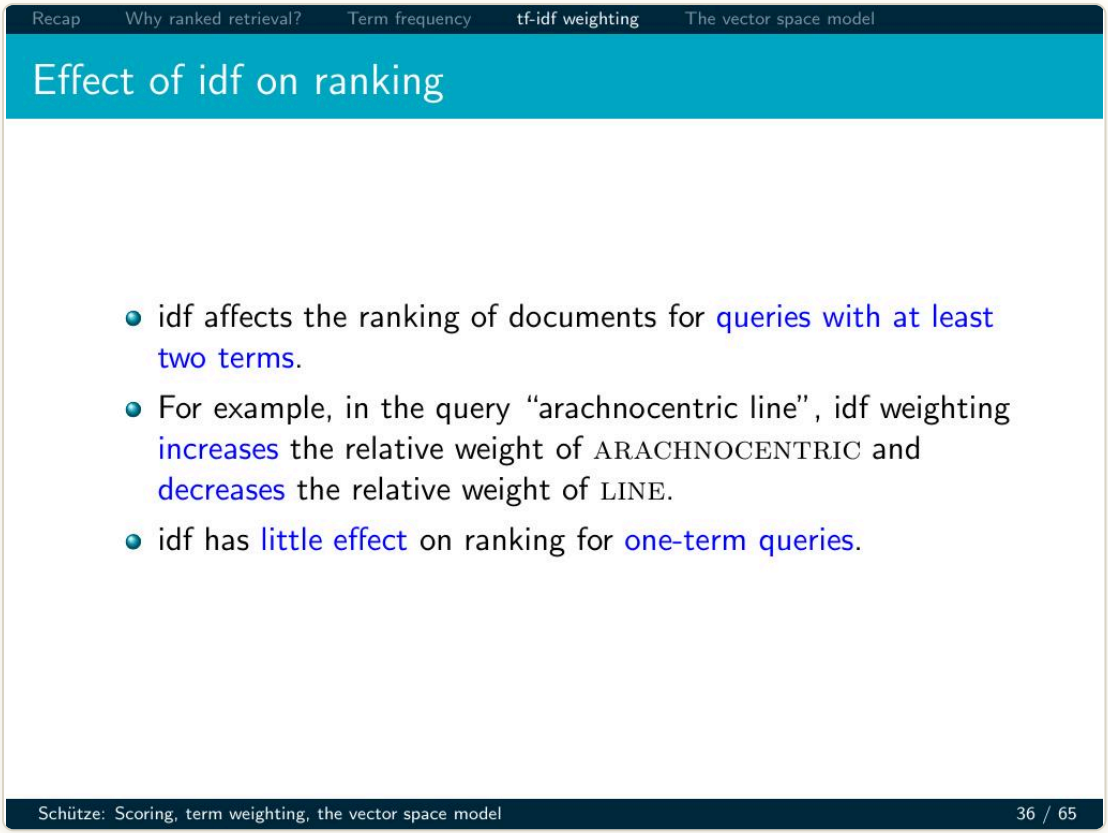
idf 예

- idf(t) = $\log_{10}$ (1,000,000 / df(t))로 계산

```
calpurnia   df 1          idf 6
animal      df 100        idf 4
sunday      df 1,000      idf 3
fly         df 10,000     idf 2
under       df 100,000    idf 1
the         df 1,000,000  idf 0
```

✍ 메모

★ 시험 모든 문서에 나오는 단어(the)의 idf = 0

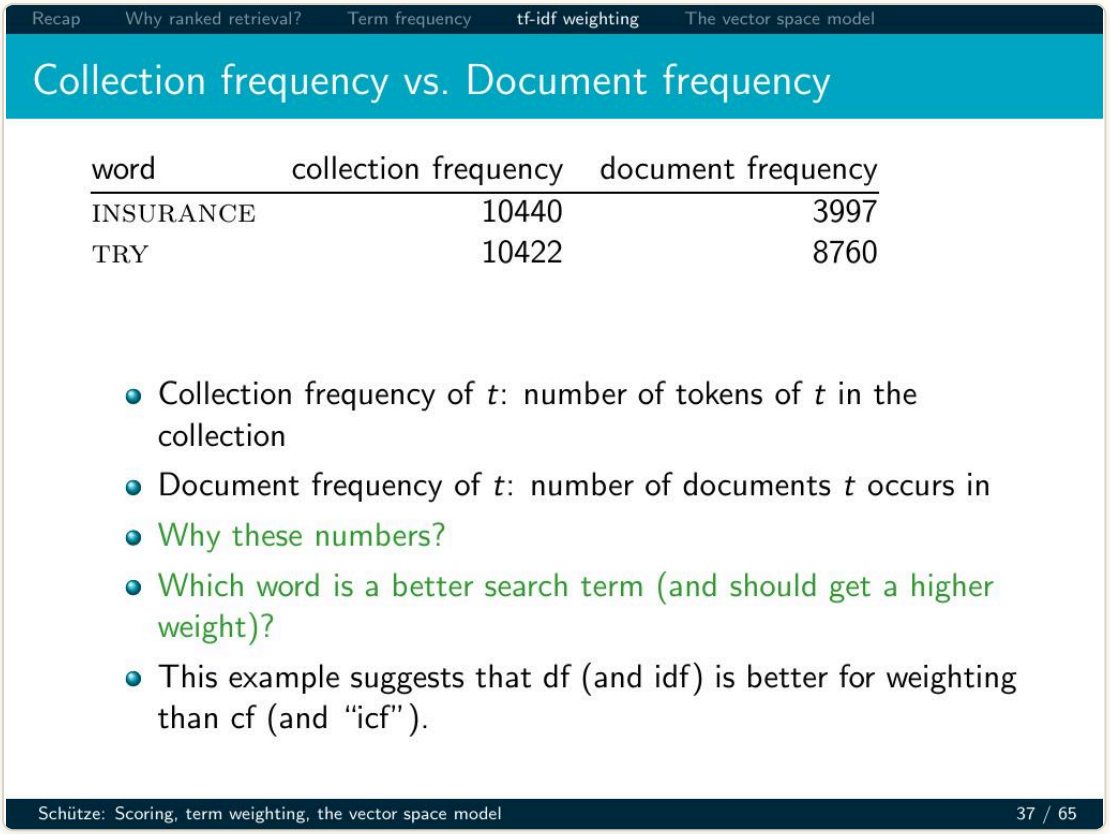


idf가 순위에 미치는 효과

- idf는 단어가 **2개 이상**인 질의에서 문서 순위에 영향을 준다
- 예: 질의 "arachnocentric line"에서 idf 가중치는 arachnocentric의 상대 가중치를 **높이고** line의 상대 가중치를 **낮춘다**
- **한 단어 질의**엔 idf가 순위에 거의 영향이 **없다**

✍ 메모

★ 시험 단어 1개 질의 → idf는 모든 문서에 같은 상수배 → 순위 불변

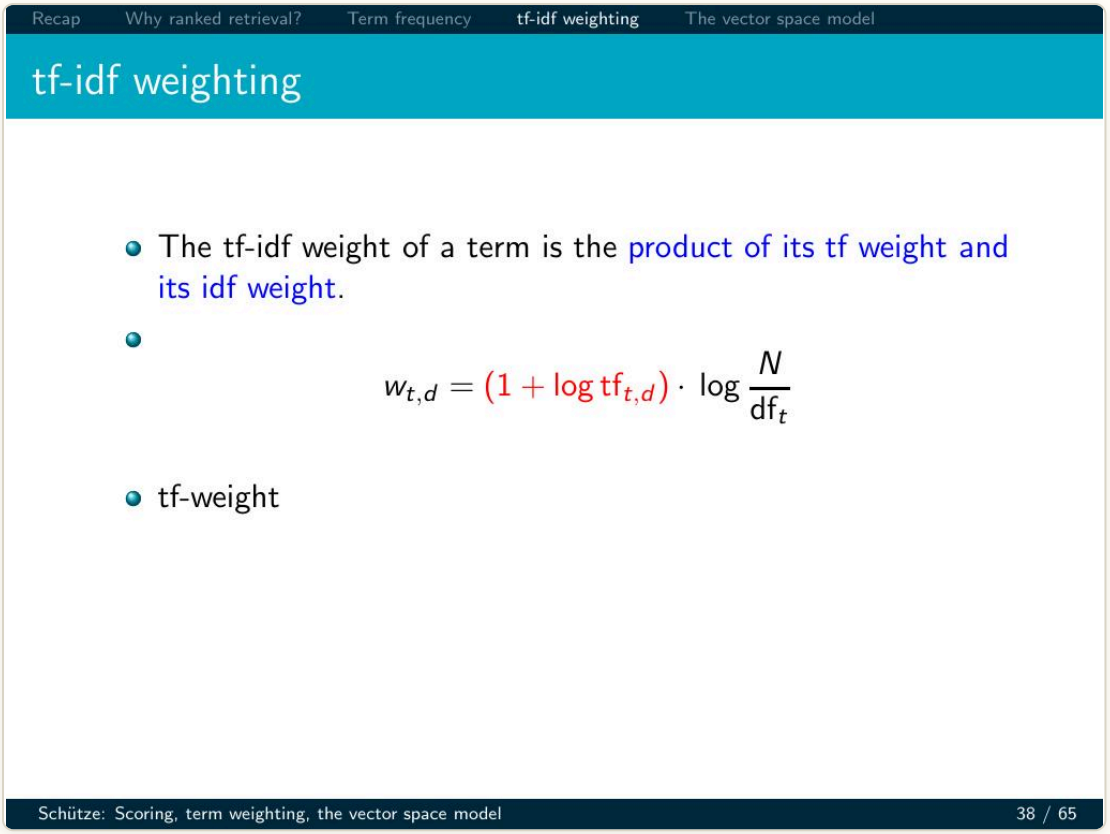


컬렉션 빈도 vs 문서 빈도

- 단어 / 컬렉션 빈도(cf) / 문서 빈도(df)
  - INSURANCE 10440 / **3997**
  - TRY 10422 / **8760**
- 단어 t의 **컬렉션 빈도**: 컬렉션에서 t의 토큰 수
- 단어 t의 **문서 빈도**: t가 나오는 문서 수
- 왜 이런 숫자가 나올까?
- 어느 단어가 더 좋은 검색어일까 (더 높은 가중치를 받아야 할까)?
- 이 예는 가중치에 cf(와 'icf')보다 **df(와 idf)**가 더 낫다는 걸 보여 준다

✍ 메모

✏ 예·풀이 cf는 비슷하지만 insurance는 적은 문서에 몰려 나옴(df 작음) → 더 좋은 검색어, idf가 더 높다

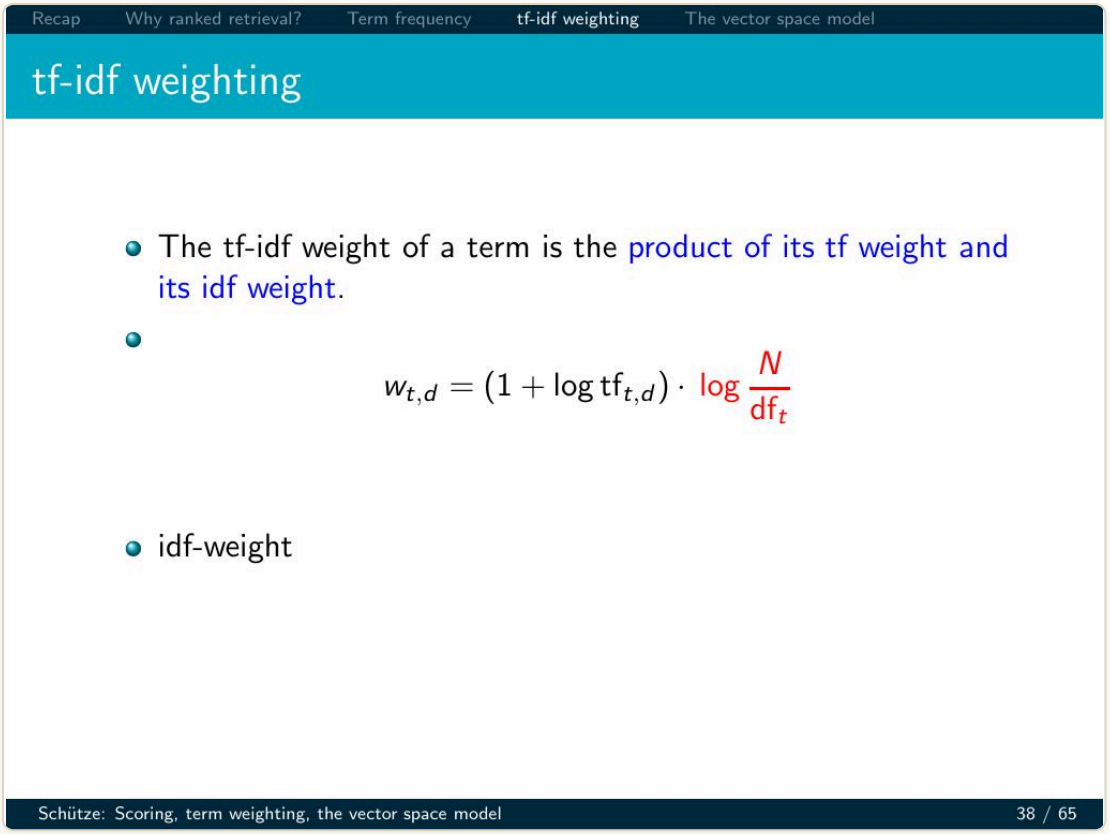


tf-idf 가중치

- 단어의 tf-idf 가중치 = **tf 가중치 × idf 가중치**
- **w(t,d) = (1 + log tf(t,d)) · log (N / df(t))**
- 앞 부분 (1 + log tf) = **tf 가중치**

✍ 메모

— 넘어가도 되는 쪽

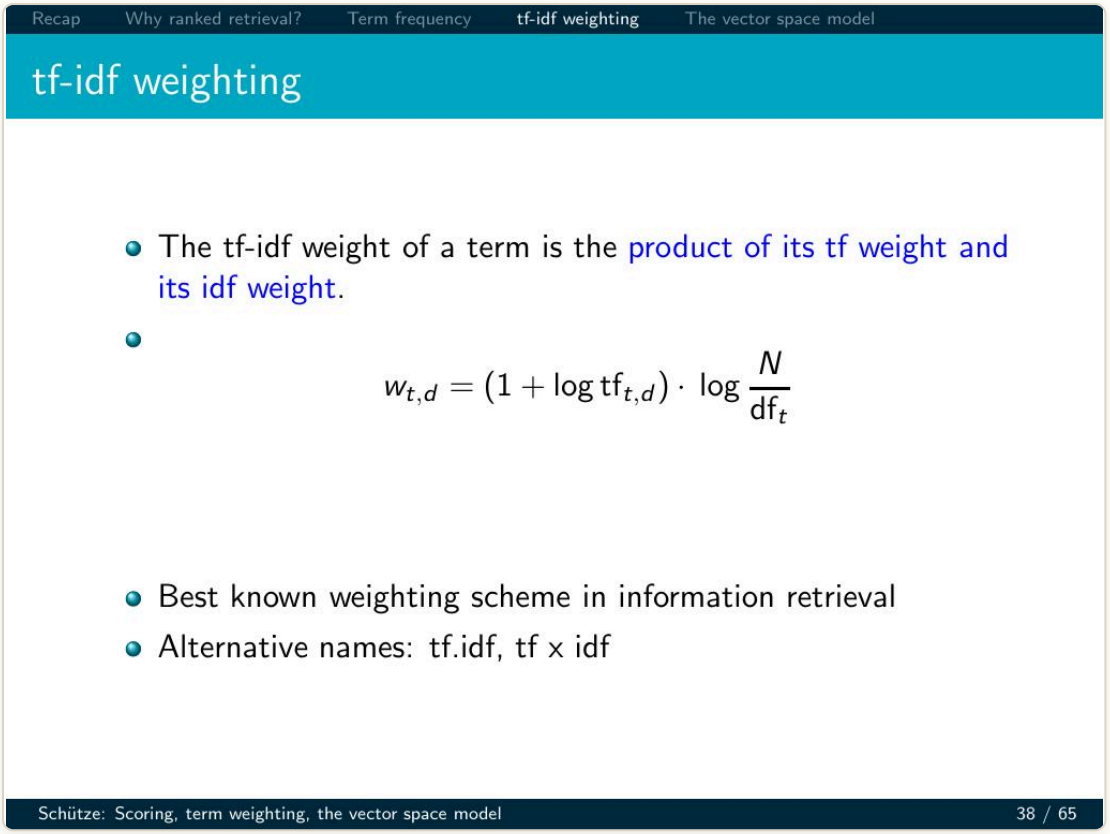


tf-idf 가중치

- 단어의 tf-idf 가중치 = **tf 가중치 × idf 가중치**
- **w(t,d) = (1 + log tf(t,d)) · log (N / df(t))**
- 뒷 부분 log N/df = **idf 가중치**

✍ 메모

— 넘어가도 되는 쪽

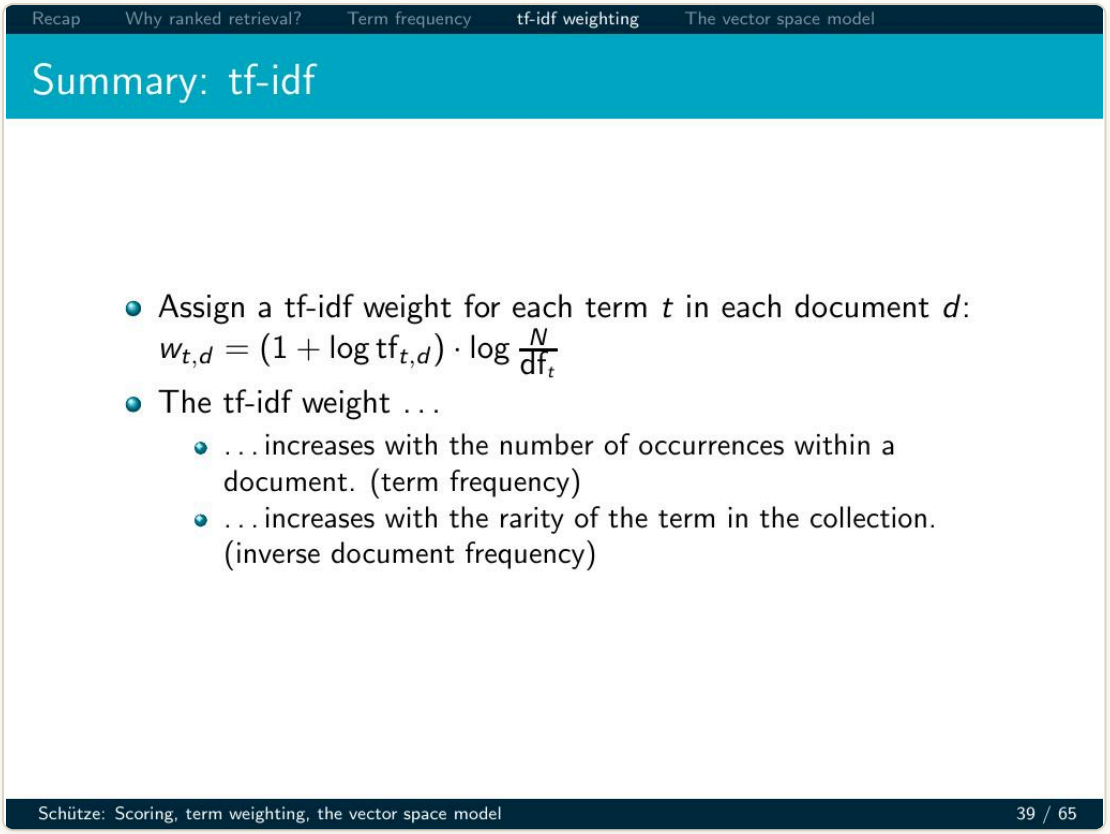


tf-idf 가중치

- 단어의 tf-idf 가중치 = **tf 가중치 × idf 가중치**
- **w(t,d) = (1 + log tf(t,d)) · log (N / df(t))**
- 정보검색에서 **가장 잘 알려진** 가중치 방식 — 다른 이름: tf.idf, tf × idf

✍ 메모

— 넘어가도 되는 쪽



요약: tf-idf

- 각 문서 d의 각 단어 t에 tf-idf 가중치를 준다: **w(t,d) = (1 + log tf(t,d)) · log (N / df(t))**
- tf-idf 가중치는 ...
  - ... 문서 안 **등장 횟수**가 많을수록 커진다 (단어 빈도)
  - ... 컬렉션에서 단어가 **드물수록** 커진다 (역문서 빈도)

✍ 메모

★ 시험 공식 그대로 외우기 + 두 성질

Exercise: Term, collection and document frequency

Quantity	Symbol	Definition
term frequency	$\mathrm{tf}_{t,d}$	number of occurrences of t in d
document frequency	df_t	number of documents in the collection that t occurs in
collection frequency	cf_t	total number of occurrences of t in the collection

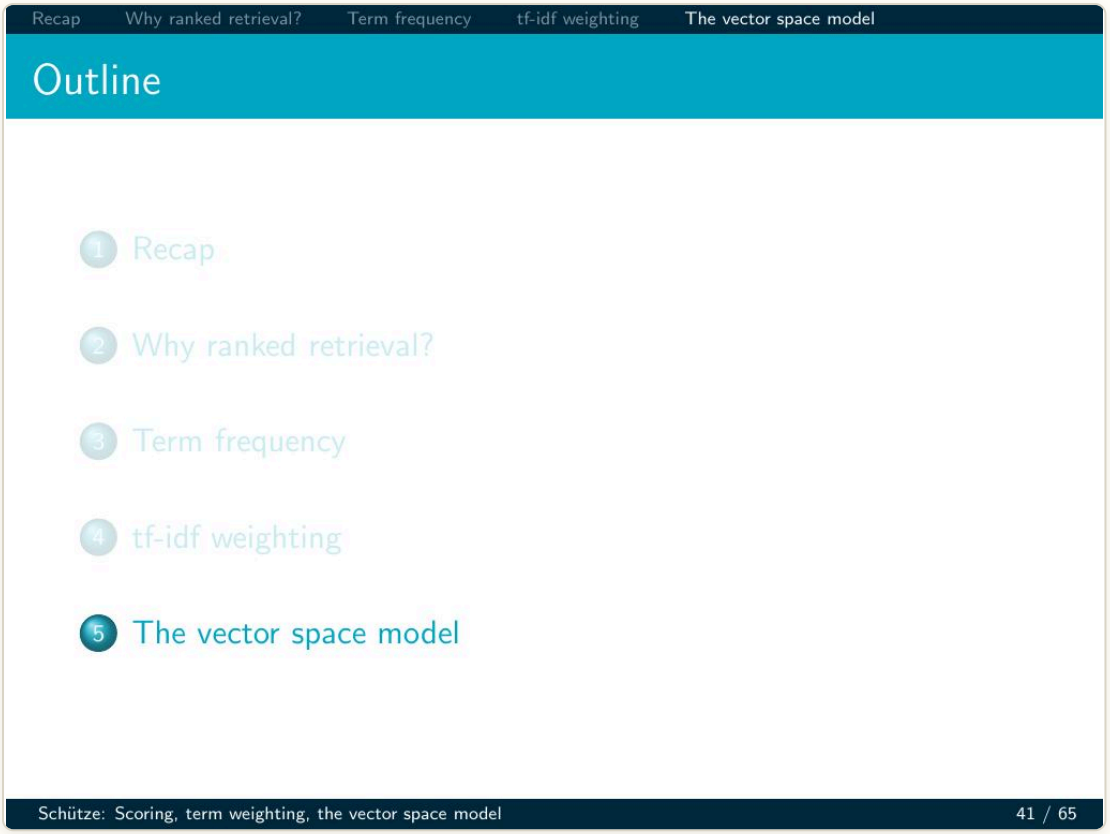
- Relationship between df and cf?
- Relationship between tf and cf?
- Relationship between tf and df?

연습: 단어 빈도, 컬렉션 빈도, 문서 빈도

- **tf(t,d)** 단어 빈도: d에서 t의 등장 횟수
- **df(t)** 문서 빈도: 컬렉션에서 t가 나오는 문서 수
- **cf(t)** 컬렉션 빈도: 컬렉션에서 t의 총 등장 횟수
- df와 cf의 관계는? tf와 cf의 관계는? tf와 df의 관계는?

✍ 메모

✏ 예·풀이 **df ≤ cf (문서마다 1번 이상 나오니까) / cf = Σ_d tf(t,d) / tf와 df는 직접 관계없음 (tf는 문서별, df는 컬렉션 전체)**



목차

- 복습 → 왜 순위 검색인가 → 단어 빈도 → tf-idf 가중치 → 벡터 공간 모델
- 지금: **벡터 공간 모델**

✍️ 메모

— 넘어가도 되는 쪽

Binary incidence matrix

	Anthony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	...
ANTHONY	1	1	0	0	0	1	
BRUTUS	1	1	0	1	0	0	
CAESAR	1	1	0	1	1	1	
CALPURNIA	0	1	0	0	0	0	
CLEOPATRA	1	0	0	0	0	0	
MERCY	1	0	1	1	1	1	
WORSER	1	0	1	1	1	0	
...							

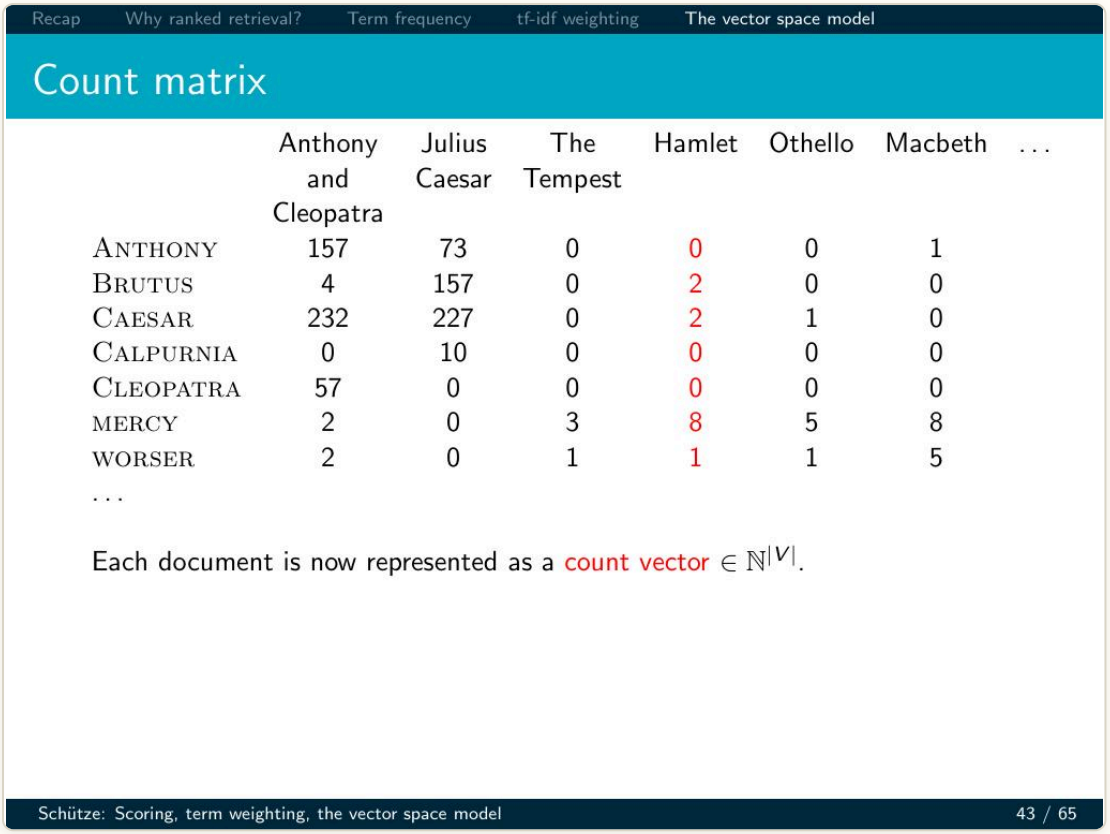
Each document is represented as a binary vector $\in \{0,1\}^{|V|}$.

이진 발생 행렬

- 각 문서를 이진 벡터 ∈ {0, 1}^|V|로 표현

✍ 메모

— 넘어가도 되는 쪽



개수 행렬

- 행 = 단어, 열 = 희곡, 칸 = 등장 횟수
  - Anthony 157 73 0 0 0 1 / Brutus 4 157 0 2 0 0 / Caesar 232 227 0 2 1 0 / Calpurnia 0 10 0 0 0 0
  - Cleopatra 57 0 0 0 0 0 / mercy 2 0 3 8 5 8 / worser 2 0 1 1 1 5
- 각 문서가 이제 **개수 벡터** ∈ ℕ^|V|

✍ 메모

— 넘어가도 되는 쪽

Recap Why ranked retrieval? Term frequency tf-idf weighting The vector space model

Binary → count → weight matrix

	Anthony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	...
ANTHONY	5.25	3.18	0.0	0.0	0.0	0.35	
BRUTUS	1.21	6.10	0.0	1.0	0.0	0.0	
CAESAR	8.59	2.54	0.0	1.51	0.25	0.0	
CALPURNIA	0.0	1.54	0.0	0.0	0.0	0.0	
CLEOPATRA	2.85	0.0	0.0	0.0	0.0	0.0	
MERCY	1.51	0.0	1.90	0.12	5.25	0.88	
WORSER	1.37	0.0	0.11	4.15	0.25	1.95	
...							

Each document is now represented as a real-valued vector of tf-idf weights $\in \mathbb{R}^{|V|}$.

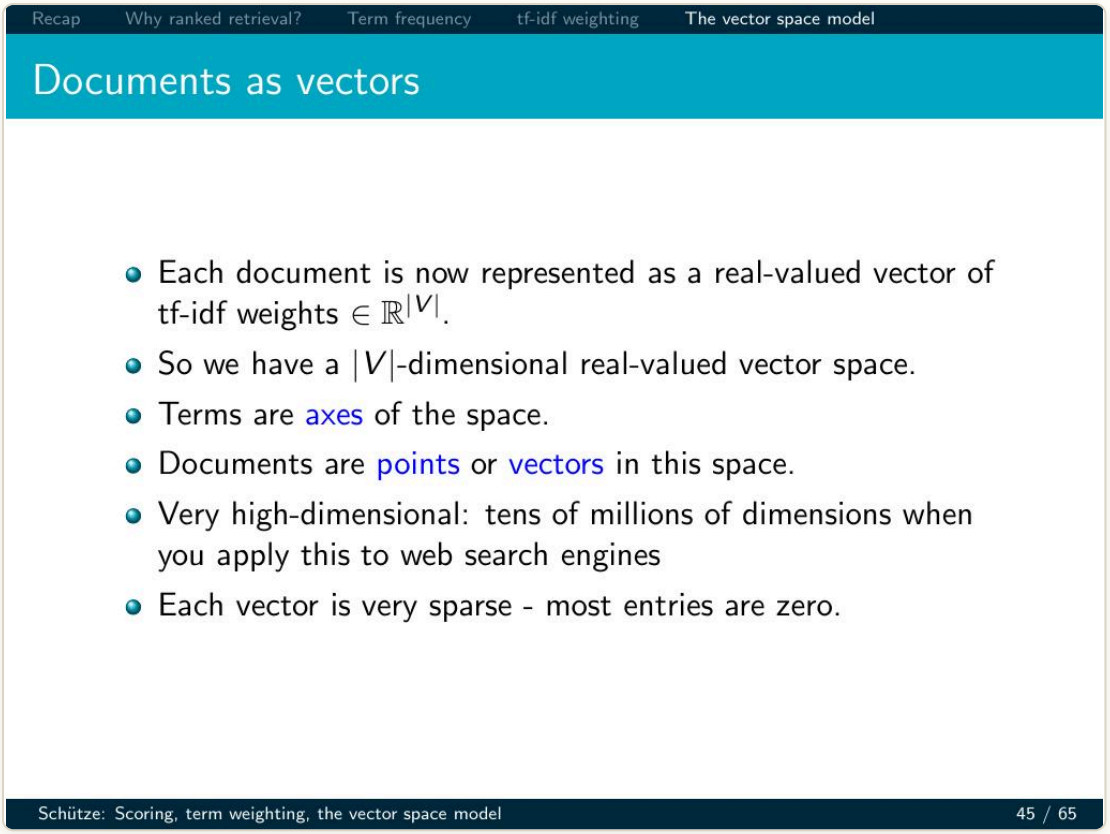
Schütze: Scoring, term weighting, the vector space model 44 / 65

이진 → 개수 → 가중치 행렬

- 칸 = **tf-idf 가중치** (예: Anthony 5.25 3.18 0.0 0.0 0.0 0.35 / Brutus 1.21 6.10 0.0 1.0 0.0 0.0 ...)
- 각 문서가 이제 tf-idf 가중치의 **실수 벡터** ∈ ℝ^|V|

✍ 메모

— 넘어가도 되는 쪽

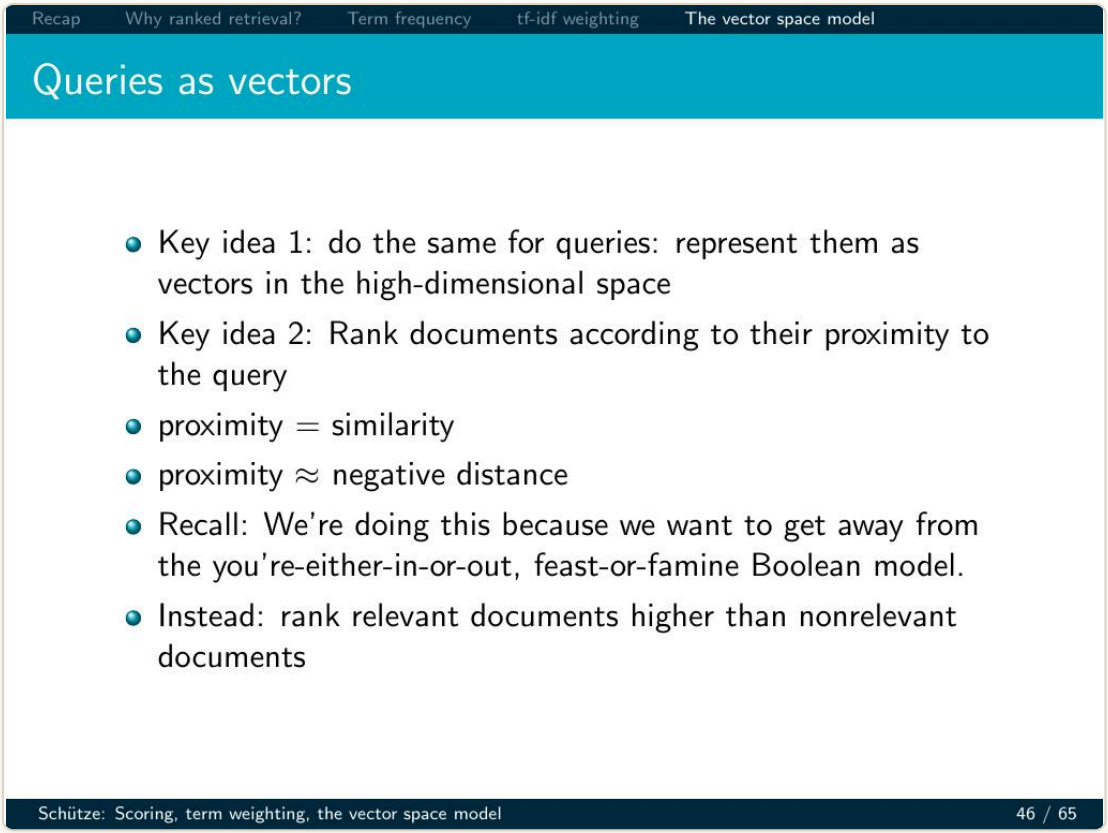


벡터로서의 문서

- 각 문서는 tf-idf 가중치의 실수 벡터 ∈ ℝ^|V|
- 그래서 **|V|차원 실수 벡터 공간**
- **단어가 축**
- **문서는 이 공간의 점(벡터)**
- 차원이 매우 높다: 웹 검색 엔진이면 **수천만 차원**
- 각 벡터는 매우 **희소**하다 — 대부분 0

✍ 메모

— 넘어가도 되는 쪽

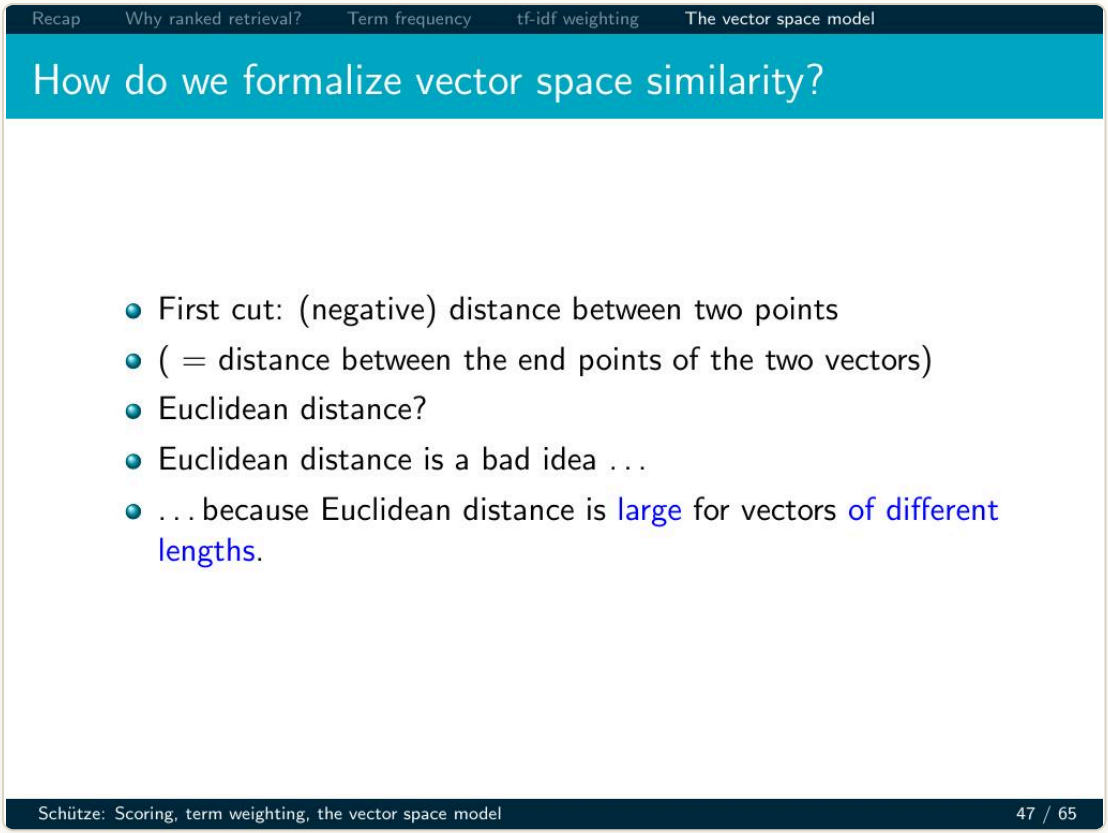


벡터로서의 질의

- 핵심 아이디어 1: **질의도 똑같이** 고차원 공간의 벡터로 표현
- 핵심 아이디어 2: 질의와의 **가까움**으로 문서 순위
  - 가까움 = **유사도**
  - 가까움 ≈ 거리의 음수
- 다시: 맞거나 안 맞거나, 잔치 아니면 굶주림인 불리언 모델에서 벗어나려는 것
- 대신: 관련 문서를 비관련 문서보다 **위에**

✍ 메모

— 넘어가도 되는 쪽

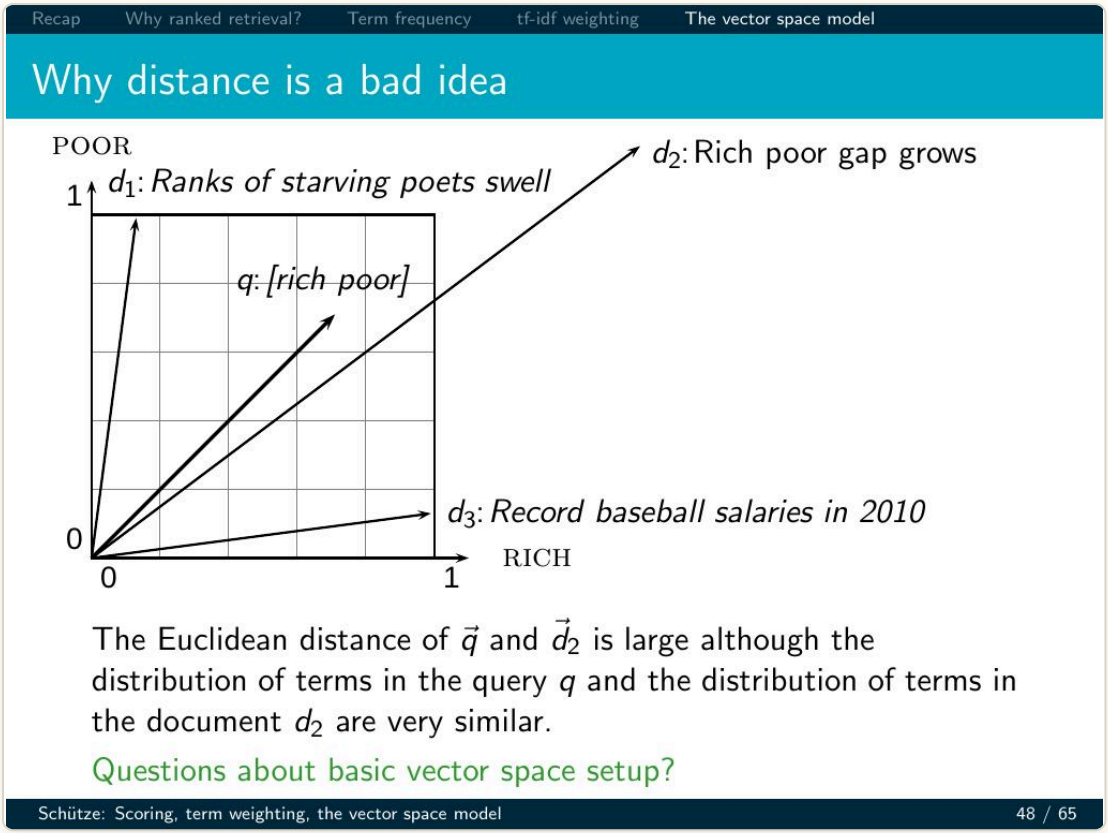


벡터 공간 유사도를 어떻게 형식화할까?

- 첫 시도: 두 점 사이의 (음의) **거리** (= 두 벡터 끝점 사이 거리)
- **유클리드 거리**?
- 유클리드 거리는 나쁜 생각 ...
- ... **길이가 다른 벡터**끼리는 유클리드 거리가 크기 때문

✍ 메모

— 넘어가도 되는 쪽

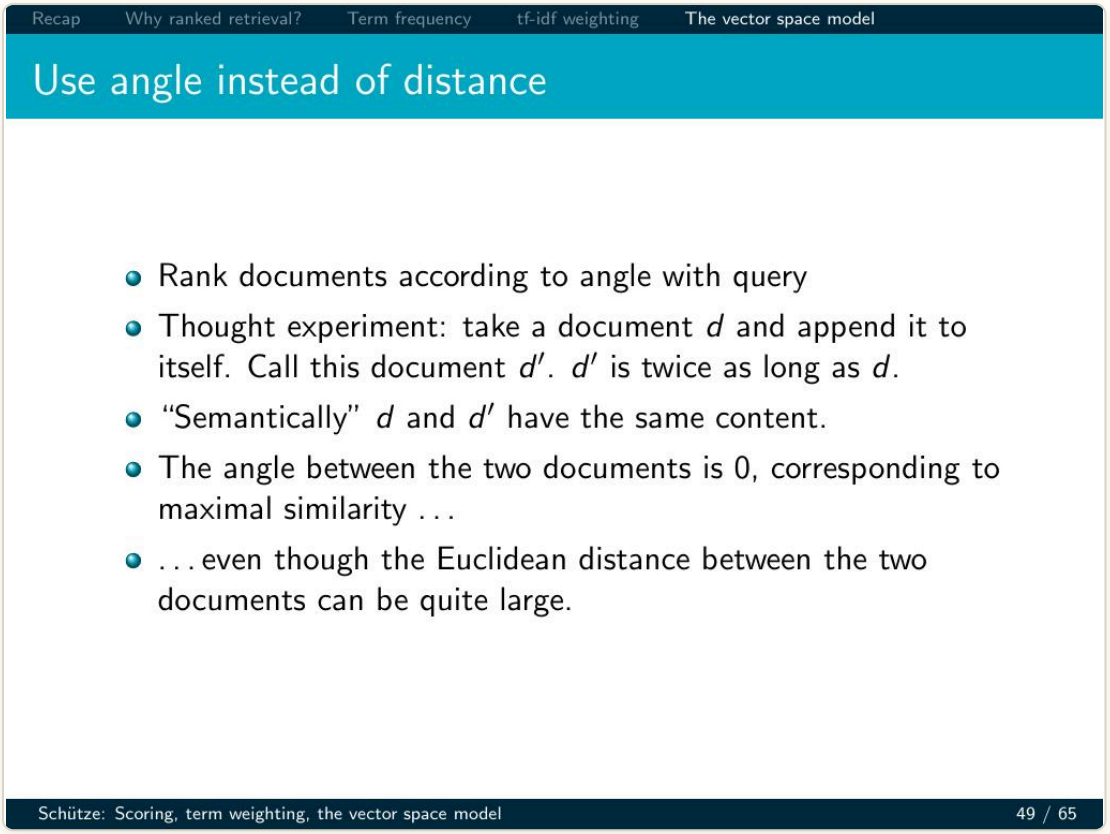


거리가 나쁜 이유 (그림)

- 축: RICH, POOR
- d1: Ranks of starving poets swell (poor 쪽) / d2: Rich poor gap grows (대각선, 길다) / d3: Record baseball salaries in 2010 (rich 쪽)
- q: [rich poor] (대각선, 짧다)
- q와 d2의 **유클리드 거리는 크지만**, 질의와 d2의 **단어 분포는 매우 비슷**하다

✍ 메모

★ 시험 방향은 같은데 길이만 다름 → 거리는 크게 나온다 → 각도를 쓰자

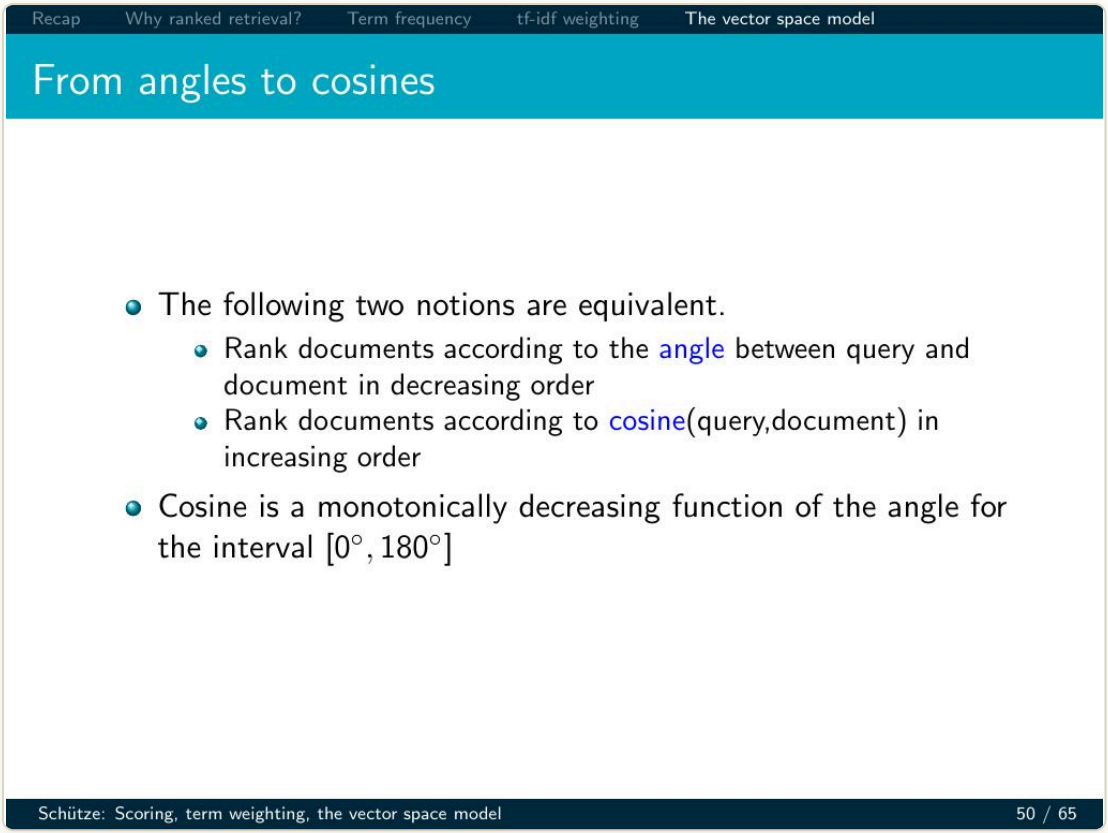


거리 대신 각도

- 질의와의 **각도**로 문서 순위
- 사고 실험: 문서 d를 자기 자신에게 이어 붙여 d′을 만든다 — d′은 d의 두 배 길이
- '의미상' d와 d′은 같은 내용
- 두 문서 사이 **각도는 0** — 최대 유사도 ...
- ... 유클리드 거리는 꽤 클 수 있는데도

✍ 메모

— 넘어가도 되는 쪽

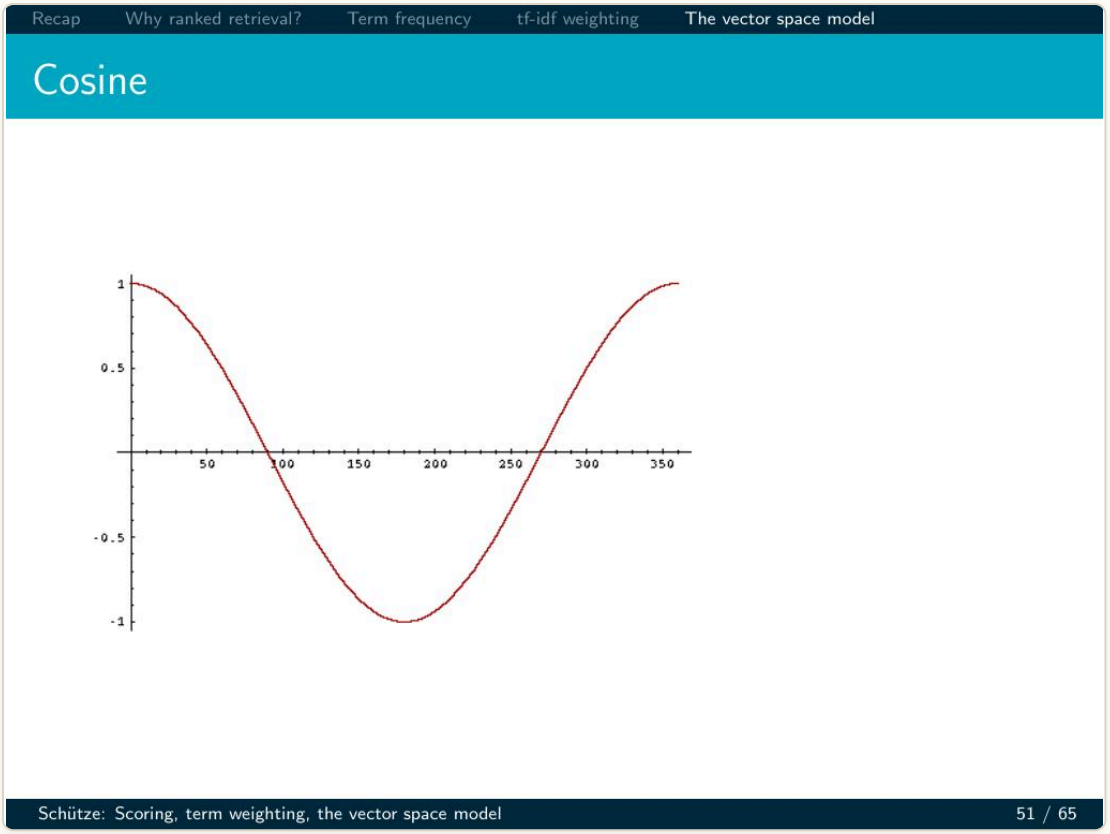


각도에서 코사인으로

- 다음 두 가지는 **같다**
  - 질의와 문서 사이 **각도의 오름차순**으로 순위 (작을수록 위)
  - **cosine(질의, 문서)의 내림차순**으로 순위 (클수록 위)
- [0°, 180°]에서 코사인은 각도의 **단조 감소 함수**

✍️ 메모

⚠ 주의 슬라이드 원문은 'angle ... decreasing / cosine ... increasing'으로 순서가 뒤바뀌어 있다. 각도는 작을수록, 코사인은 클수록 유사

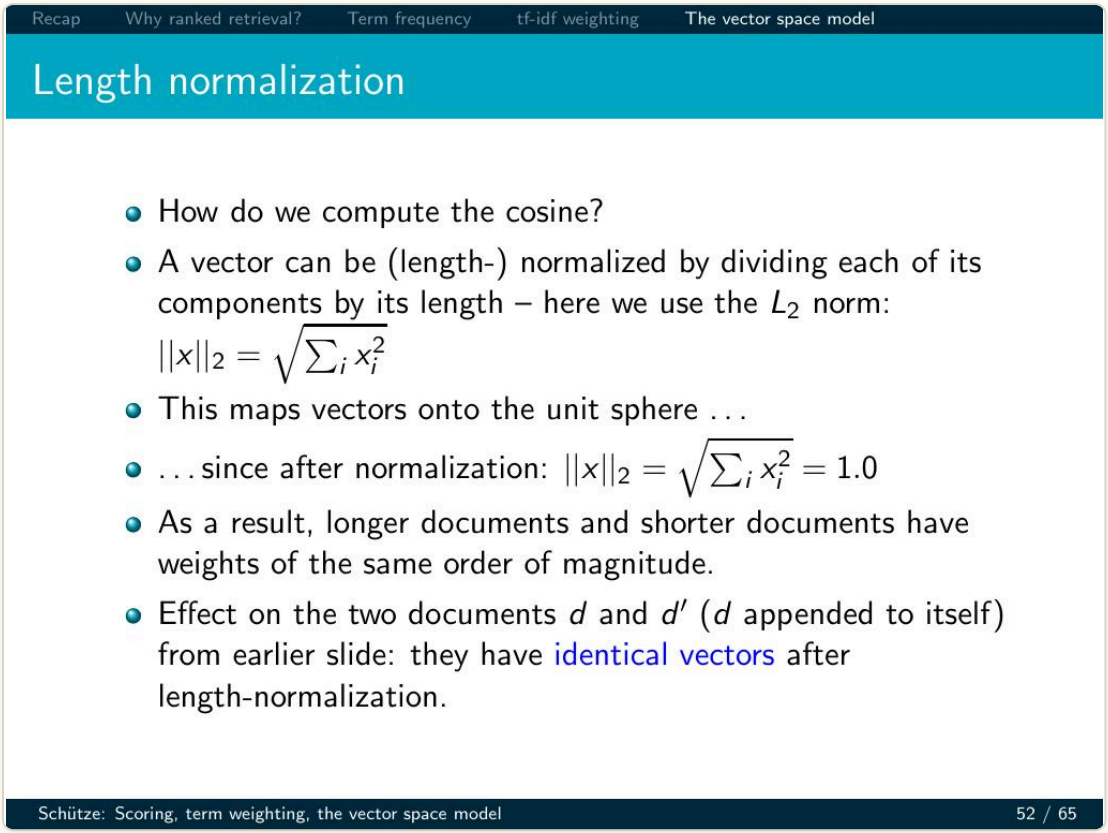


코사인 (그래프)

- cos 그래프: 0°에서 1, 90°에서 0, 180°에서 −1 — 0~180° 구간에서 단조 감소

✍ 메모

— 넘어가도 되는 쪽



길이 정규화

- 코사인을 어떻게 계산하나?
- 벡터의 각 성분을 **길이로 나눠** (길이) 정규화 — 여기선 **L2 노름**: $\|x\|_2 = \sqrt{(\Sigma\, x_i^2)}$
- 벡터를 **단위 구** 위로 옮긴다 ($\|x\|_2 = 1.0$)
- 그래서 긴 문서와 짧은 문서의 가중치가 **같은 규모**가 된다
- 앞의 d와 d′(자기 자신을 이어 붙인 것): 길이 정규화 후 **똑같은 벡터**

✍ 메모

— 넘어가도 되는 쪽

Cosine similarity between query and document

$$\cos(\vec{q},\vec{d}) = \text{SIM}(\vec{q},\vec{d}) = \frac{\vec{q}\cdot\vec{d}}{|\vec{q}||\vec{d}|} = \frac{\sum_{i=1}^{|V|} q_i d_i}{\sqrt{\sum_{i=1}^{|V|} q_i^2}\sqrt{\sum_{i=1}^{|V|} d_i^2}}$$

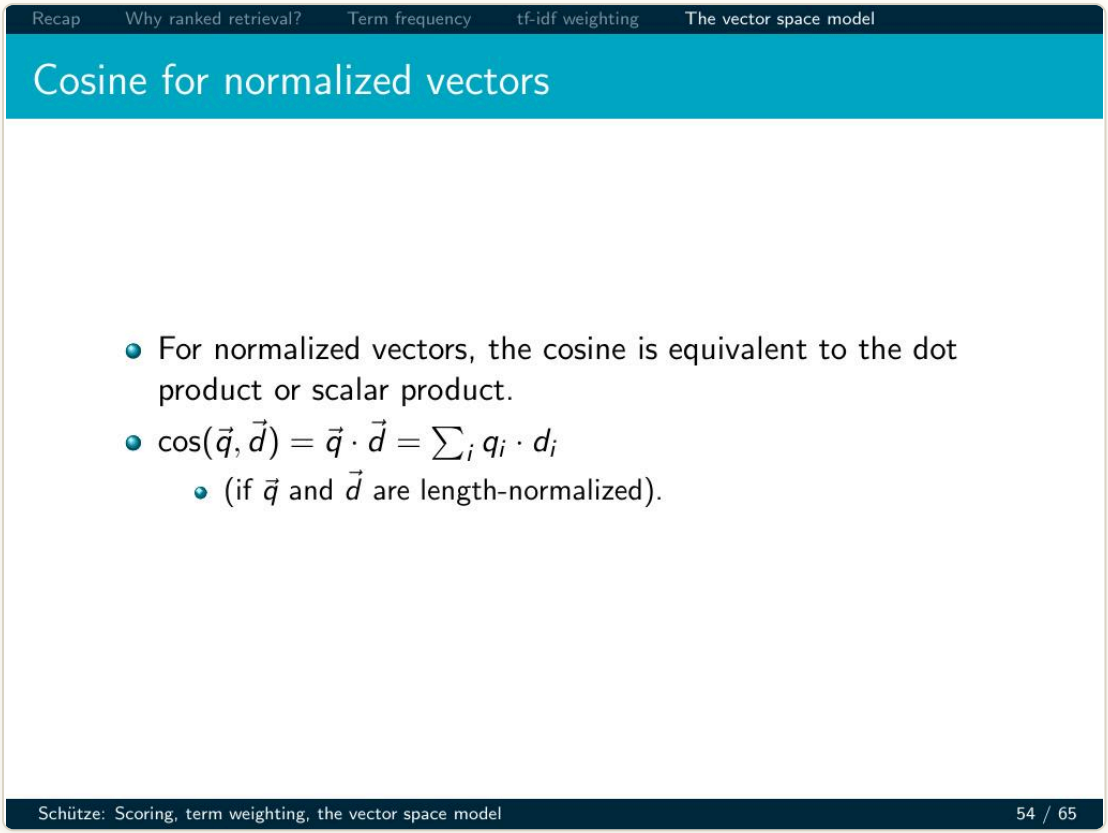
- q_i is the tf-idf weight of term i in the query.
- d_i is the tf-idf weight of term i in the document.
- $|\vec{q}|$ and $|\vec{d}|$ are the lengths of $\vec{q}$ and $\vec{d}$.
- This is the cosine similarity of $\vec{q}$ and $\vec{d}$ . . . . . or, equivalently, the cosine of the angle between $\vec{q}$ and $\vec{d}$.

질의와 문서의 코사인 유사도

- **$\cos(q, d) = \text{SIM}(q, d) = (q \cdot d) / (|q|\,|d|) = \Sigma\, q_i d_i / (\sqrt{\Sigma q_i^2} \cdot \sqrt{\Sigma d_i^2})$**
- q_i = 질의에서 단어 i의 tf-idf 가중치
- d_i = 문서에서 단어 i의 tf-idf 가중치
- |q|와 |d| = q와 d의 길이
- 이것이 q와 d의 **코사인 유사도** ... 또는 q와 d 사이 각도의 코사인

✍ 메모

★ 시험 공식 그대로 — 계산 문제 확실



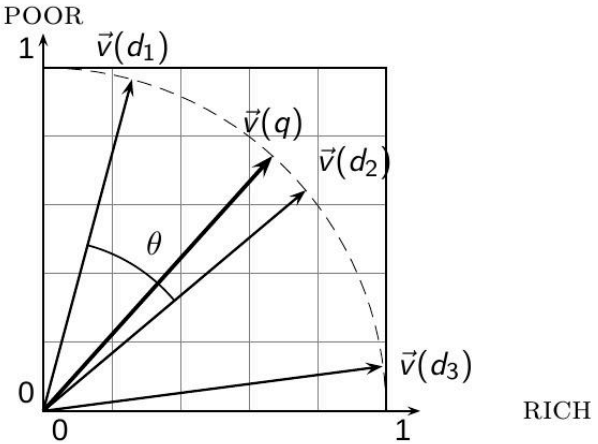
정규화된 벡터의 코사인

- 정규화된 벡터에선 코사인 = **내적(dot product)**
- **$\cos(q, d) = q \cdot d = \Sigma\, q_i \cdot d_i$** (q와 d가 길이 정규화됐을 때)

✍ 메모

— 넘어가도 되는 쪽

Cosine similarity illustrated

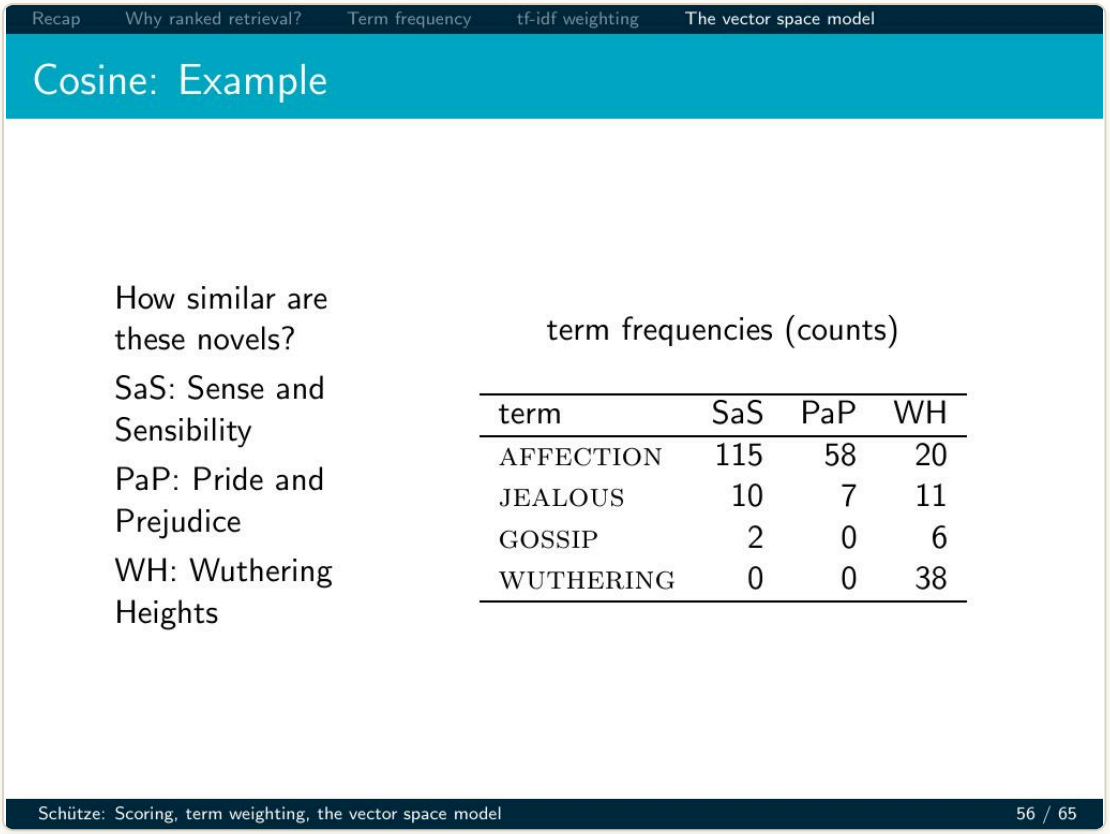


코사인 유사도 (그림)

- RICH-POOR 평면에서 v(q)와 v(d_2) 사이 각도 θ가 작다 → 유사
- v(d_1), v(d_3)는 질의와 각도가 크다

✍ 메모

— 넘어가도 되는 쪽

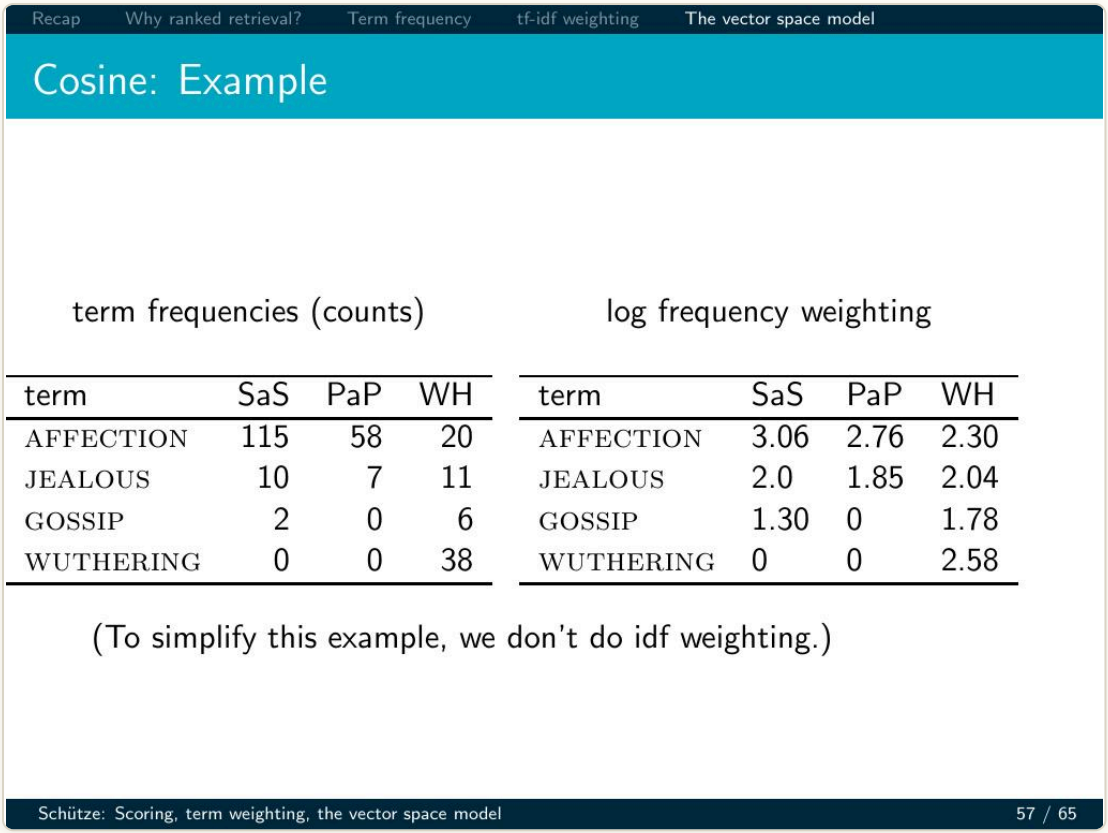


코사인: 예

- 이 소설들은 얼마나 비슷할까?
  - SaS: Sense and Sensibility / PaP: Pride and Prejudice / WH: Wuthering Heights
- 단어 빈도(개수): SaS / PaP / WH
  - AFFECTION 115 / 58 / 20
  - JEALOUS 10 / 7 / 11
  - GOSSIP 2 / 0 / 6
  - WUTHERING 0 / 0 / 38

✍ 메모

— 넘어가도 되는 쪽



코사인: 예 — 로그 빈도 가중치

- 로그 빈도 가중치 (1 + log tf)
  - AFFECTION 3.06 / 2.76 / 2.30
  - JEALOUS 2.0 / 1.85 / 2.04
  - GOSSIP 1.30 / 0 / 1.78
  - WUTHERING 0 / 0 / 2.58
- (예를 단순하게 하려고 **idf 가중치는 안 한다**)

✍ 메모

✏ 예·풀이 115 → 1 + log 115 = 1 + 2.06 = 3.06

Recap Why ranked retrieval? Term frequency tf-idf weighting The vector space model

Cosine: Example

log frequency weighting			
term	SaS	PaP	WH
AFFECTION	3.06	2.76	2.30
JEALOUS	2.0	1.85	2.04
GOSSIP	1.30	0	1.78
WUTHERING	0	0	2.58

log frequency weighting & cosine normalization			
term	SaS	PaP	WH
AFFECTION	0.789	0.832	0.524
JEALOUS	0.515	0.555	0.465
GOSSIP	0.335	0.0	0.405
WUTHERING	0.0	0.0	0.588

- cos(SaS,PaP) ≈ $0.789 * 0.832 + 0.515 * 0.555 + 0.335 * 0.0 + 0.0 * 0.0 \approx 0.94$.
- cos(SaS,WH) ≈ 0.79
- cos(PaP,WH) ≈ 0.69
- Why do we have cos(SaS,PaP) > cos(SAS,WH)?

Schütze: Scoring, term weighting, the vector space model 58 / 65

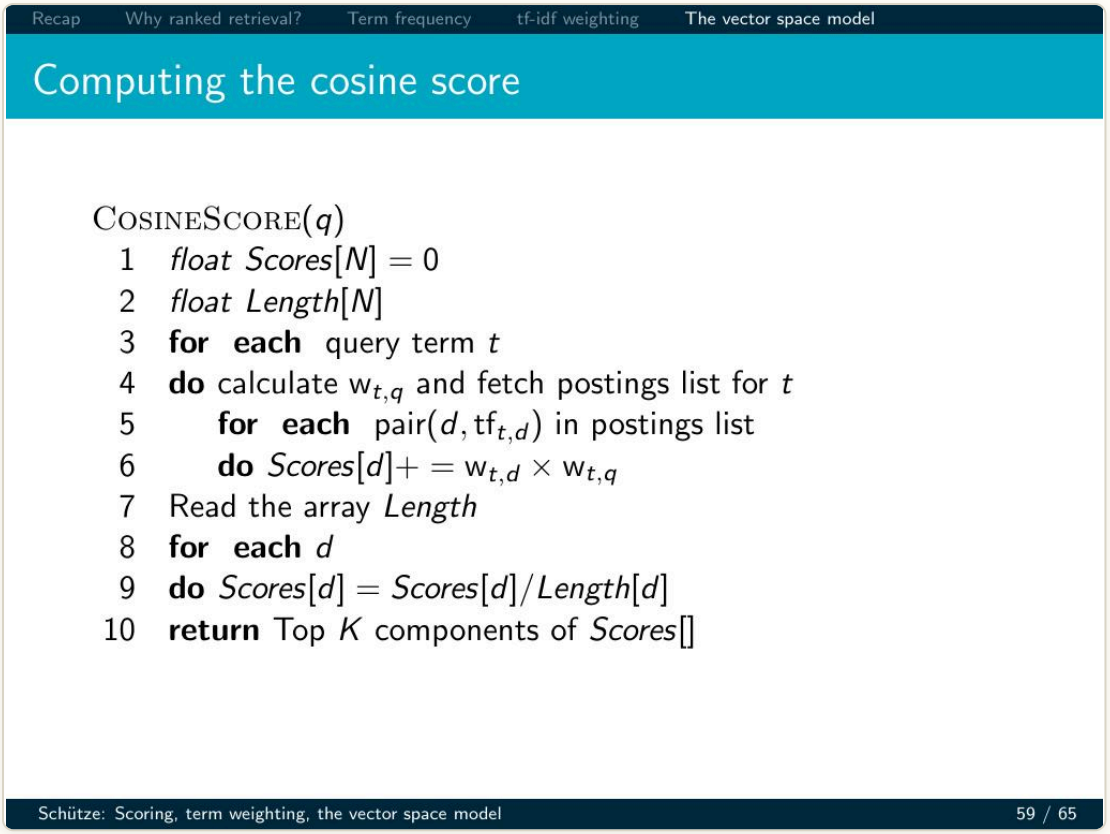
코사인: 예 — 코사인 정규화

- 로그 빈도 + 코사인 정규화
  - AFFECTION 0.789 / 0.832 / 0.524
  - JEALOUS 0.515 / 0.555 / 0.465
  - GOSSIP 0.335 / 0.0 / 0.405
  - WUTHERING 0.0 / 0.0 / 0.588
- cos(SaS, PaP) ≈ 0.789 × 0.832 + 0.515 × 0.555 + 0.335 × 0.0 + 0.0 × 0.0 ≈ **0.94**
- cos(SaS, WH) ≈ **0.79**
- cos(PaP, WH) ≈ **0.69**
- 왜 cos(SaS, PaP) > cos(SaS, WH)일까?

✍ 메모

✏ 예·풀이 SaS 길이 = √(3.06² + 2.0² + 1.30²) ≈ 3.88 → 3.06/3.88 = 0.789

💡 이해 WH에만 wuthering이 많고, SaS·PaP는 affection 중심이라 방향이 비슷

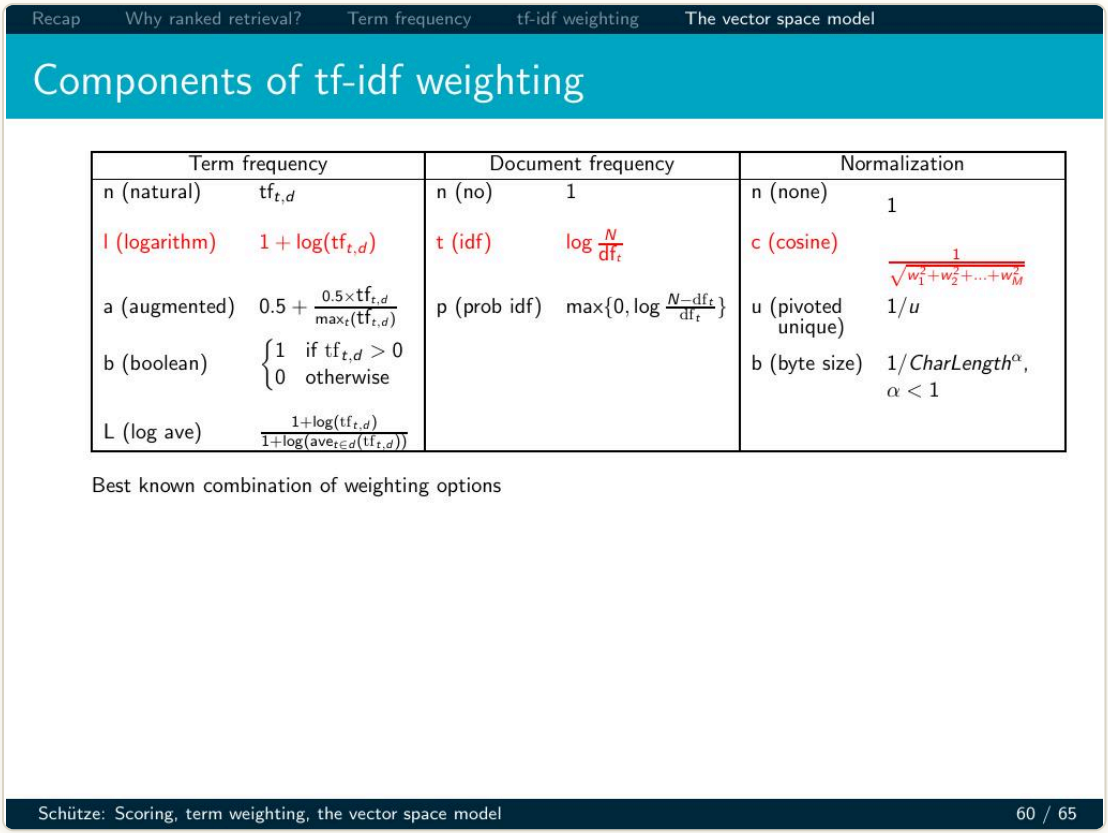


코사인 점수 계산 COSINESCORE(q)

```
float Scores[N] = 0; float Length[N]
for each 질의어 t:
  w(t,q)를 계산하고 t의 포스팅 리스트를 가져온다
  for each (d, tf(t,d)) in 포스팅 리스트: Scores[d] += w(t,d) × w(t,q)
Length 배열을 읽는다
for each d: Scores[d] = Scores[d] / Length[d]
return Scores[]의 상위 K개
```

✍️ 메모

💡 이해 단어 단위로 점수를 누적(term-at-a-time), 마지막에 문서 길이로 나눔



tf-idf 가중치의 구성 요소 (SMART 표기)

- **단어 빈도**: n(natural) tf / **l(logarithm) 1 + log tf** / a(augmented) 0.5 + 0.5·tf/max tf / b(boolean) tf>0이면 1 / L(log ave)
- **문서 빈도**: n(no) 1 / **t(idf) log N/df** / p(prob idf) max{0, log (N−df)/df}
- **정규화**: n(none) 1 / **c(cosine) 1/√(w₁² + … + w_M²)** / u(pivoted unique) / b(byte size)
- **가장 잘 알려진 조합** (빨간색: l, t, c)

✍ 메모

★ 시험 세 글자 = tf · df · 정규화. 자주 나오는 글자: n, l / n, t / n, c

Components of tf-idf weighting

Term frequency		Document frequency		Normalization	
n (natural)	$\mathrm{tf}_{t,d}$	n (no)	1	n (none)	1
l (logarithm)	$1+\log(\mathrm{tf}_{t,d})$	t (idf)	$\log \frac{N}{\mathrm{df}_t}$	c (cosine)	$\frac{1}{\sqrt{w_1^2+w_2^2+\ldots+w_M^2}}$
a (augmented)	$0.5+\frac{0.5\times\mathrm{tf}_{t,d}}{\max_t(\mathrm{tf}_{t,d})}$	p (prob idf)	$\max\{0,\log\frac{N-\mathrm{df}_t}{\mathrm{df}_t}\}$	u (pivoted unique)	$1/u$
b (boolean)	$\begin{cases}1 & \text{if } \mathrm{tf}_{t,d}>0\\ 0 & \text{otherwise}\end{cases}$			b (byte size)	$1/CharLength^{\alpha}$, $\alpha<1$
L (log ave)	$\frac{1+\log(\mathrm{tf}_{t,d})}{1+\log(\mathrm{ave}_{t\in d}(\mathrm{tf}_{t,d}))}$				

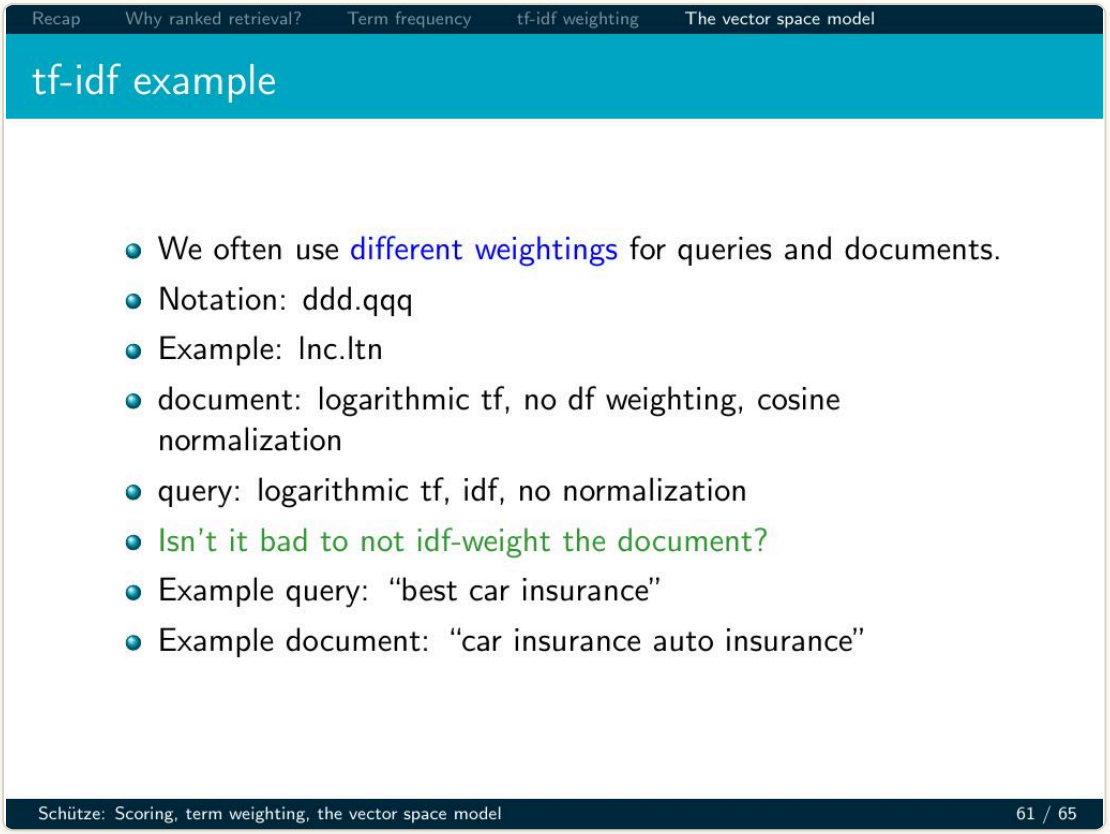
Default: no weighting

tf-idf 가중치의 구성 요소 — 기본값

- 같은 표, 기본값 = **가중치 없음** (n, n, n)

✍ 메모

— 넘어가도 되는 쪽

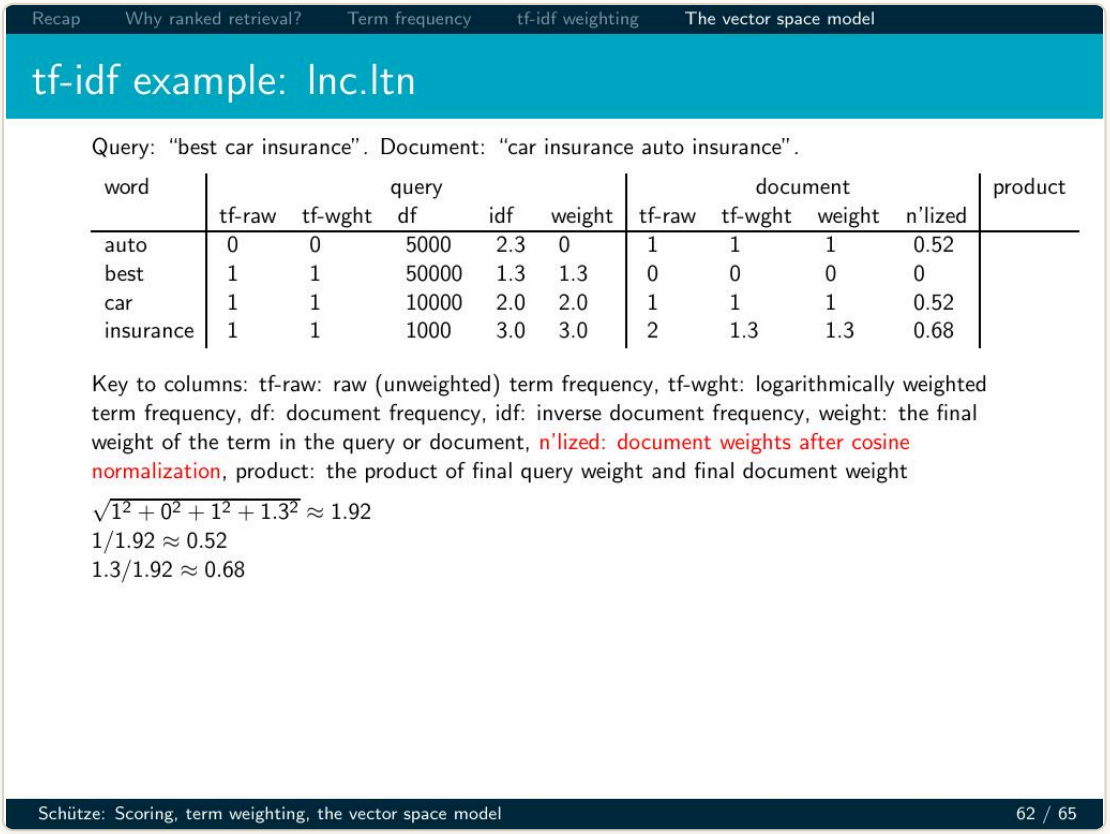


tf-idf 예

- 질의와 문서에 **다른 가중치**를 쓰는 경우가 많다
- 표기: **ddd.qqq** (앞 3글자 = 문서, 뒤 3글자 = 질의)
- 예: **lnc.ltn**
  - 문서: 로그 tf, df 가중치 없음, 코사인 정규화
  - 질의: 로그 tf, idf, 정규화 없음
- 문서에 idf를 안 주는 게 나쁘지 않을까?
- 예 질의: "best car insurance"
- 예 문서: "car insurance auto insurance"

✍ 메모

★ 시험 lnc.ltn: 문서 = l n c / 질의 = l t n



Recap Why ranked retrieval? Term frequency tf-idf weighting The vector space model

tf-idf example: lnc.ltn

Query: "best car insurance". Document: "car insurance auto insurance".

word	query					document				product
	tf-raw	tf-wght	df	idf	weight	tf-raw	tf-wght	weight	n'lized	
auto	0	0	5000	2.3	0	1	1	1	0.52	
best	1	1	50000	1.3	1.3	0	0	0	0	
car	1	1	10000	2.0	2.0	1	1	1	0.52	
insurance	1	1	1000	3.0	3.0	2	1.3	1.3	0.68	

Key to columns: tf-raw: raw (unweighted) term frequency, tf-wght: logarithmically weighted term frequency, df: document frequency, idf: inverse document frequency, weight: the final weight of the term in the query or document, n'lized: document weights after cosine normalization, product: the product of final query weight and final document weight

$\sqrt{1^2 + 0^2 + 1^2 + 1.3^2} \approx 1.92$
$1/1.92 \approx 0.52$
$1.3/1.92 \approx 0.68$

Schütze: Scoring, term weighting, the vector space model 62 / 65

tf-idf 예: lnc.ltn (표)

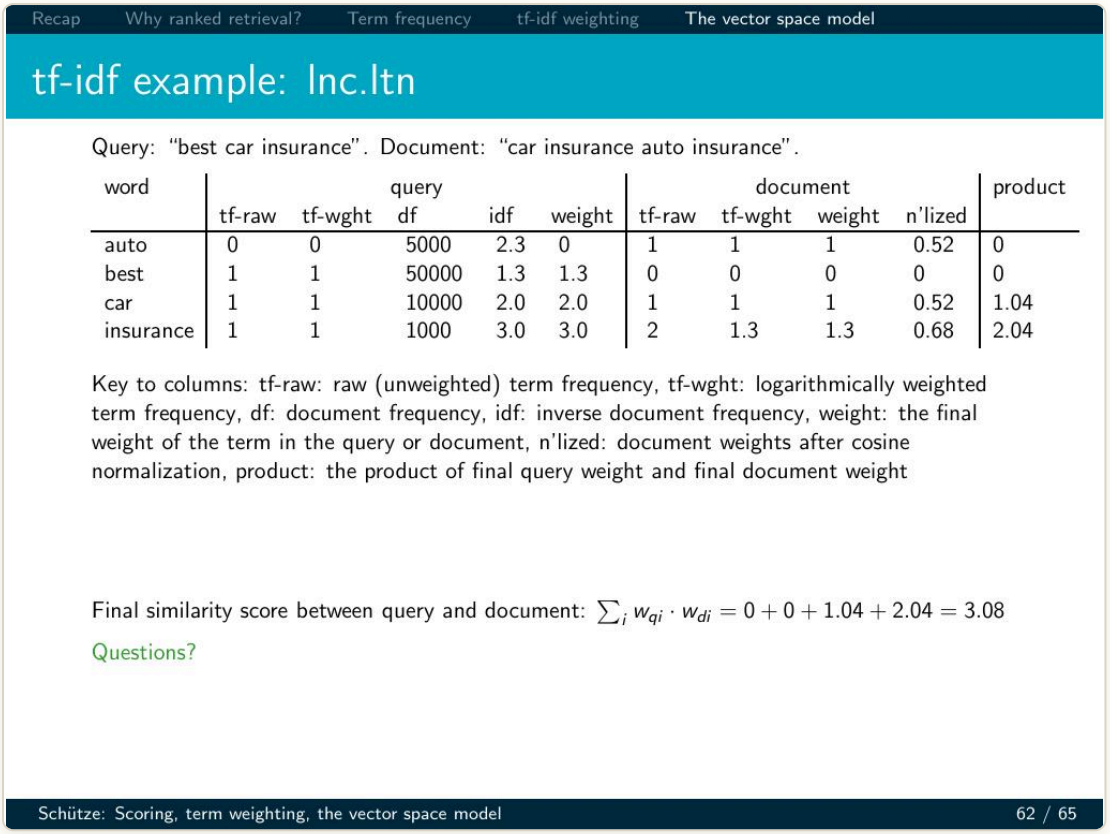
- 단어 / 질의(tf-raw, tf-wght, df, idf, weight) / 문서(tf-raw, tf-wght, weight, n'lized) / 곱

```
auto       0 0 5000  2.3 0   | 1 1   1   0.52 | 0
best       1 1 50000 1.3 1.3 | 0 0   0   0    | 0
car        1 1 10000 2.0 2.0 | 1 1   1   0.52 | 1.04
insurance  1 1 1000  3.0 3.0 | 2 1.3 1.3 0.68 | 2.04
```

- 문서 길이: √(1² + 0² + 1² + 1.3²) ≈ **1.92**
- 1/1.92 ≈ 0.52, 1.3/1.92 ≈ 0.68

✍ 메모

✏ 예·풀이 idf: N = 1,000,000 가정 → car $\log(10^6/10^4) = 2.0$, insurance $\log(10^6/10^3) = 3.0$



Recap Why ranked retrieval? Term frequency tf-idf weighting The vector space model

tf-idf example: lnc.ltn

Query: "best car insurance". Document: "car insurance auto insurance".

word	query					document				product
	tf-raw	tf-wght	df	idf	weight	tf-raw	tf-wght	weight	n'lized	
auto	0	0	5000	2.3	0	1	1	1	0.52	0
best	1	1	50000	1.3	1.3	0	0	0	0	0
car	1	1	10000	2.0	2.0	1	1	1	0.52	1.04
insurance	1	1	1000	3.0	3.0	2	1.3	1.3	0.68	2.04

Key to columns: tf-raw: raw (unweighted) term frequency, tf-wght: logarithmically weighted term frequency, df: document frequency, idf: inverse document frequency, weight: the final weight of the term in the query or document, n'lized: document weights after cosine normalization, product: the product of final query weight and final document weight

Final similarity score between query and document: $\sum_i w_{qi} \cdot w_{di} = 0 + 0 + 1.04 + 2.04 = 3.08$

Questions?

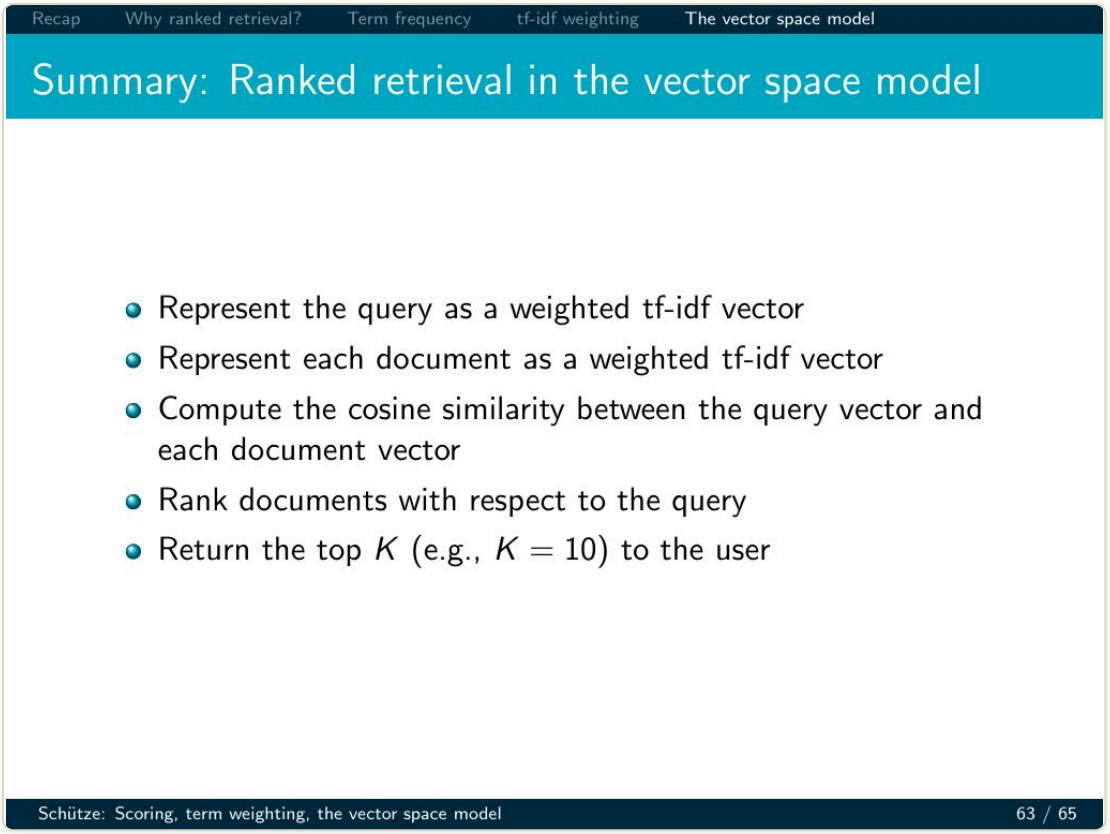
Schütze: Scoring, term weighting, the vector space model 62 / 65

tf-idf 예: lnc.ltn (최종 점수)

- 질의와 문서의 최종 유사도 점수: **Σ w(q,i) · w(d,i) = 0 + 0 + 1.04 + 2.04 = 3.08**

✍ 메모

★ 시험 답 3.08 — 손계산 문제 그대로 나올 수 있음

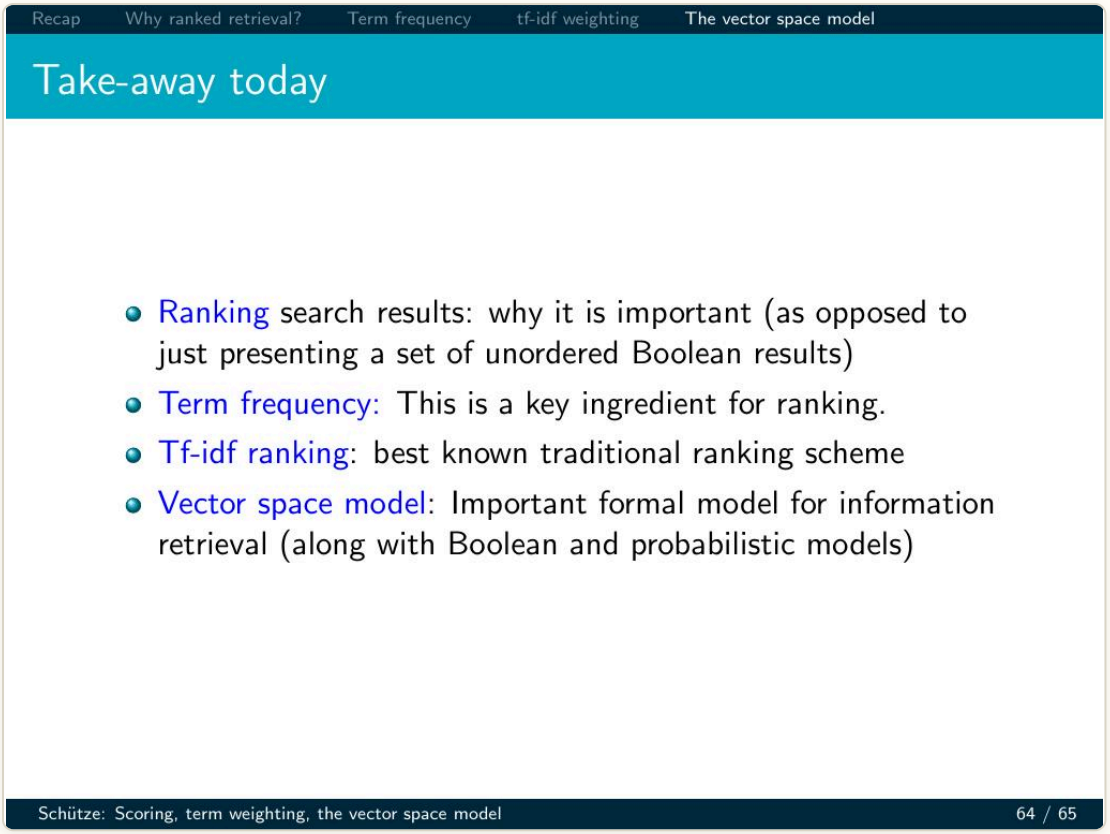


요약: 벡터 공간 모델의 순위 검색

- 질의를 가중 tf-idf **벡터**로 표현
- 각 문서를 가중 tf-idf 벡터로 표현
- 질의 벡터와 각 문서 벡터의 **코사인 유사도** 계산
- 질의에 대해 문서 **순위**
- 상위 **K**개(예: K = 10)를 사용자에게

✍ 메모

— 넘어가도 되는 쪽

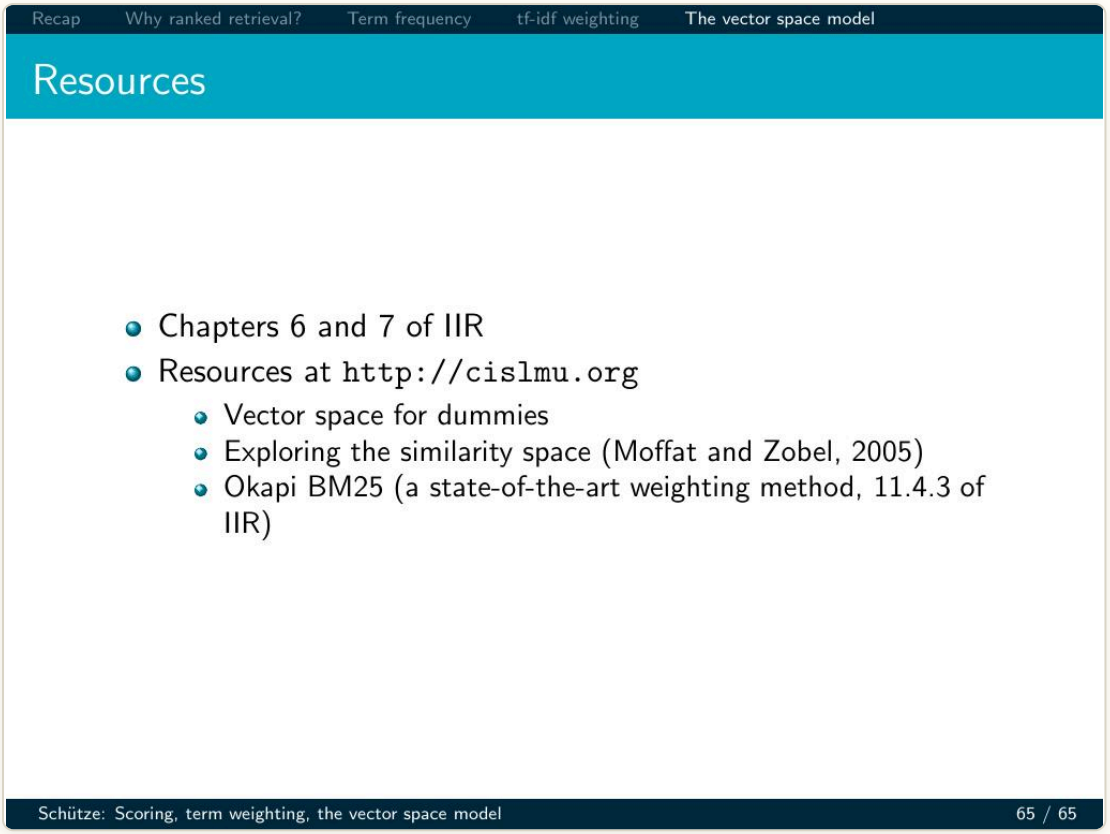


오늘 가져갈 것

- 순위화가 왜 중요한가
- 단어 빈도: 순위화의 핵심 재료
- tf-idf 순위화: 가장 잘 알려진 전통적 방식
- 벡터 공간 모델: 불리언·확률 모델과 함께 IR의 중요한 형식 모델

✍ 메모

— 넘어가도 되는 쪽



참고 자료

- IIR 6·7장, http://cislmu.org
- Vector space for dummies
- Exploring the similarity space (Moffat & Zobel, 2005)
- Okapi BM25 (최신 가중치 방법, IIR 11.4.3)

✍ 메모

— 넘어가도 되는 쪽